SAINT MICHAEL COLLEGE OF CARAGA

SEMANTIC TEXT AND SOURCE CODE AI CHECKER USING A HYBRID MODEL FOR ACADEMIC ASSESSMENT

A Thesis Presented To
The faculty of
College of Computing and Information Sciences

In Partial Requirements for the Degree
BACHELOR OF SCIENCE IN COMPUTER SCIENCE

By

RIO R. BARCELON

RHASSAB M. SUBSUBAN

ALMER B. BONGCAHIG

SUSAN M. PAGULA

ADVISER: **LEALIL I. PALACIO**

March 2026

TABLE OF CONTENTS

		Pages
TITLE PAGE		i
CERTIFICATE OF RESEARCH APPROVAL		ii
ABSTRACT		iii
DEDICATION		iv
ACKNOWLEDGEMENT		v
TABLE OF CONTENTS		vi
LIST OF TABLES		viii
LIST OF FIGURES		x
CHAPTER		
1	**INTRODUCTION**	1
	1.1 Project Context	1
	1.2 Purpose and Description	2
	1.3 Objective of the Study	4
	1.4 Scope and Limitations	4
	1.5 Definition of Terms	5
2	**REVIEW OF RELATED LITERATURE**	19
3	**TECHNICAL BACKGROUND**	9
	3.1 System Architecture	9
	3.2 Conceptual Diagram	10
	3.3 Use Case Diagram	12
	3.4 Sequence Diagram	13
	3.5 ERD (Database Design)	14
	3.6 User Interface Design (Prototype)	
	3.7 Software Platforms, Development Environments and Tools	17
	3.8 Hardware Requirements	
	3.9 Evaluation Matrix	
4	**SOFTWARE DEVELOPMENT AND TESTING**	19
	4.1 Development Process	19
	4.2 Testing Process	19
	- Accuracy Test	
	- Functionality Test	
	- Performance Test	
5	**SUMMARY, CONCLUSIONS, AND RECOMMENDATIONS**	44
	5.1 Summary of Findings	45
	5.2 Conclusion	46
	5.3 Recommendations	47
REFERENCES		50
CURRICULUM VITAE		59



APPENDICES
CERTIFICATE OF TECHNOLOGY-BASED ASSESSMENT 68
PUBLISHABLE JOURNAL 100

SAINT MICHAEL COLLEGE OF CARAGA
College Of Computing and Information Sciences

CERTIFICATE OF RESEARCH APPROVAL

This thesis is entitled **"** Click here to enter text. **"** presented and submitted by Click here to enter text. In partial fulfillment of the requirements for the degree of Click here to enter text. Is hereby accepted and recommended for Oral Examination.

LEALIL I. PALACIO
Adviser

RIO R. BARCELON
Member

RHASSAB M. SUBSUBAN
Member

ALMER B. BONGCAHIG
Member

SUSAN M. PAGULA
Member

Click here to enter text.
Chairperson

ACCEPTED as partial fulfillment of the requirements for the degree of Click here to enter text.

APPROVED by the tribunal at the Oral Examination with the grade of _____.

Click here to enter text.
Adviser

Click here to enter text.
Member

Click here to enter text.
Member

Click here to enter text.
Member

Click here to enter text.
Member

Click here to enter text.
Chairperson



COLLEGE OF TEACHER EDUCATION

ABSTRACT

TITLE:	Semantic Text and Source Code AI Checker Using a Hybrid Model for Academic Assessment
AUTHOR:	Group 2
DEGREE:	
ADVISER:	Lealil I. Palacio, MSIT (CAR)
PLACE OF PUBLICATION:	
DATE	
PAGES	

I. OBJECTIVES

The primary focus of this research is to create, experiment with, and enhance a hybrid model algorithm that can recognize AI-Generated text and code in academic output, helping to maintain academic honesty and reduce students' dependency on AI tools. The study will detect AI-Generated Content and develop a hybrid AI model algorithm that can identify AI-Generated text and source code in different types of academic output by combining semantic text analysis and code structure Recognition. Test the Model's Accuracy will check how well the model can find AI-Generated content in text and code. The research will also match the model's score with other tools used to detect AI content. Promote academic honesty: Look at how the model can help students focus on creating their own work, Instead of depending too much on AI to do the job for them.

Make it easy to use: Build an easy-to-use assistant for instructors to quickly spot and check AI-Generated text and code in students' output. Consider honest use: Look at the unfamiliar side

effects surrounding using AI-Generated tools in education and deliver standards for their fair and responsible use. It's not a punishment but a discipline for every student who continually abuses An AI tool.

II. METHODOLOGY

To develop and evaluate the hybrid AI detection model, this study will adhere to a well-defined and structured plan. The following steps will be part of the process:

1.1 Research Design

This study utilized a Developmental and Experimental Research Design. This approach was chosen because the primary goal was to first create and implement a new System," SEMANTIC TEXT AND SOURCE CODE AI CHECKER USING A HYBRID MODEL FOR ACADEMIC ASSESSMENT," and then scientifically evaluate its performance.

The research was structured in two distinct phases:

Phase 1: System Development

This initial phase focused on building the "TruthScan AI" platform, following the software development methodology detailed in Chapter 3. Key development activities included:

Architectural and Database Design: We defined the system's core structure, including the front-end user interface, the back-end server, and the database. An entity-relationship diagram (ERD) was created to manage user, admin, and results data.

Model and Interface Implementation: The core hybrid algorithm was built by integrating several models, including a RoBERTa/GPT classifier, and Hybrid Stylometric + AST Structural Analysis. Finally, the user interface (UI) was designed for both the main checker and the administrative dashboard.

Phase 2: Experimental Evaluation and Testing

Following development, the system underwent rigorous experimental testing to measure its effectiveness, accuracy, and performance. This evaluation was based on three specific tests:

Accuracy Test: This was the core experiment to assess the model's ability to accurately distinguish between human-written and AI-generated content. This involved preparing a validated dataset of text and code, processing it through the system, and measuring accuracy rates (including false positives and negatives). These results were then compared to the performance of other detection tools.

Functionality Test: This test ensured all system components worked as intended, including user functions like "Paste Text or Code" and "Upload File", as well as admin features like "User Monitoring".

Performance Test: This test measured non-functional requirements, such as the system's processing speed for analysis and its ability to handle inputs of varying lengths.

The Hybrid Detection Model

The system employs a hybrid model algorithm that integrates distinct technical workflows for semantic text analysis and structural code recognition.

Semantic Text Detection

Textual submissions (essays, reports) are processed through a RoBERTa/GPT classifier.

1. Semantic Analysis: The system breaks text into semantic tokens to interpret context and meaning beyond simple word matching.

2. Perplexity Calculation: The model calculates a Perplexity Score, which measures the mathematical predictability of word sequences. AI-generated text typically exhibits lower perplexity compared to human writing.

3. Final Text Score: The output is a percentage probability indicating the likelihood of AI authorship.

Source Code Detection

Programming submissions are analyzed using a combination of Abstract Syntax Tree (AST) and Hybrid Stylometric Analysis (HSA).

1. **AST Structural Mapping**: The code is parsed into an AST to map its underlying logical flow. This allows the system to detect similarity even if variable names or comments are altered.

2. **Stylometric Fingerprinting**: The system evaluates unique "coding fingerprints," such as indentation habits and function organization, to distinguish between human and AI coding patterns.

3. Final Code Score: The system generates a Similarity Index () representing the match between the logic of the submission and known AI code structures.

Evaluation Matrix and Scoring

The effectiveness of the model is validated through an **Accuracy Test** using a confusion matrix to tally raw results.

Manual Data Categorization

To determine system reliability, results are manually tallied as follows:

True Positive (TP): AI-generated content correctly flagged as AI.

False Positive (FP): Original human work mistakenly flagged as AI.

False Negative (FN): AI-generated content mistakenly identified as human.

True Negative (TN): Original human work correctly identified as human.

Statistical Formulas

The final performance metrics, including the F1 score of 88.86% achieved in this study, are calculated using the following formulas:

1. Precision: TP / TP × FP
2. Recall: TP / TP + FN
3. F1 Score: 2 × (Precision × Recall) / (Precision + Recall)

Data Example (Experimental Result)

In a controlled test of 100 submissions (50 human, 50 AI), the system's performance is represented by the following data points:

METRIC	SAMPLE TALLY	PERCENTAGE
True Positive	44	88%
False Positive	5	10%
False Negative	6	12%
Final F1 Score		88.86%

This computational framework ensures that the TruthScan AI platform provides a reliable, evidence-based assessment of academic integrity.



III. FINDINGS

IV. RECOMMENDATIONS

Keywords: Keywords

AI Detection

Academic Integrity

Hybrid Model

Text Analysis

Code Detection

Semantic Analysis

Machine Learning

AI-Generated Content

Ethical Use

Academic Assessment



DEDICATION

Our family's love and support for us have been our compass during this long process of making this thesis. We dedicated this work to them. This accomplishment would not have been possible without their full support of us. To our siblings, thank you for always being there to listen, encourage, and make us laugh during the most difficult times. Without our siblings, finishing the process would be boring and uneasy. This thesis is also dedicated to our partners and close friends, whose understanding, full support, and patience enabled us to get through late nights and long hours. Your unwavering confidence in our abilities kept us going even in the most challenging times. Last, we dedicate this thesis to our one true God, whose direction, fortitude, and grace have continuously inspired us to work harder. We have overcome the obstacles and accomplished this milestone thanks to his constant love that never fails, wisdom, and blessings. This accomplishment is equally His and ours. All glory belongs to you alone, God.



ACKNOWLEDGEMENT

We sincerely thank everyone who helped and encouraged us as we have worked tirelessly to finish this thesis. We want to start by sincerely thanking our advisor, Lealil I. Palacio, MSLIT (CAR). For their unwavering support, wisdom, and priceless guidance. Your enlightening comments, corrections, and helpful critiques have significantly influenced the course of our study and aided in our intellectual and personal development. We consider ourselves grateful to have had the chance to work under your care. Additionally, we would like to express our sincere gratitude to the College of Computing and Information Sciences faculty and staff for their support and resources, enabling us to complete our research.

We particularly appreciate the CCIS instructor's guidance and encouragement. We also want to express our deep appreciation to our families, whose love, support, encouragement, and kindness have kept us grounded and motivated throughout this entire process of making this research. To our parents, for their unwavering sacrifices, encouragement, compassion, and belief in us, and to our siblings and partners, for their understanding and encouragement during long hours of making our research and writing. Without your constant support, we could not have made it this far. Additionally, we would like to thank our friends, classmates, and co-researchers for their unwavering encouragement, criticism, and moral support. We appreciate your goodness and kindness. Your organization enhanced this experience, whether it was through



brainstorming sessions, late-night study sessions through messenger calls, or just listening. We want to thank everyone who participated in our study, gave their time and expertise, and helped make this research possible. Without your help, this thesis would not have been possible.

Finally, we thank God for His guidance, grace, and strength throughout this journey. Despite the negativity, He keeps reminding us that we don't need to work on this research on our own strength. He keeps reminding us that we must ask for His divine grace, knowledge, and understanding before anything else. With his help, we have been able to break through obstacles, persevere through hardship, and ultimately reach this achievement.



LIST OF TABLES

Table	Pages
1	
2	
3	



LIST OF FIGURES

Figure		Pages
1	System Architecture	
2	Conceptual Diagram	
3	Use Case Diagram	
4	Sequence Diagram	
5	Entity-Relationship Diagram	
6	TruthScan AI landing page	
7	*Paste Text or Code button & function.*	
8	Processing result for Processing text	

9	Paste Text or Code button & function.
10	Upload file button & function.
11	Processing result
12	AI text detection scoring
13	Paste Text or Code button & function.
14	Processing result
15	AI source code detection scoring
16	TruthScan AI footer section
17	
18	
19	

CHAPTER 1

INTRODUCTION

1.1 Project Context

These days, students use AI tools on the internet to do their assignments quickly and easily, and teachers struggle to monitor their students' work. Educational institutions increasingly use automated systems in the digital age to assess student work, including written and programming tasks. Due to their Limitations in understanding logical structures and semantic meaning, traditional plagiarism checkers and code similarity tools often fall short of providing correct assessments. This project suggests using a hybrid model created specifically for The Academic evaluation to develop a Semantic Text and Source Code AI Checker to address this issue. The proposed system combines code similarity detection algorithms with Natural language Processing (NLP) and Machine Learning (ML) approaches to provide a more reliable and accurate assessment tool. By collaborating structural and functional analysis of source codes with semantic analysis of written academic texts, the hybrid model allows the system to recognize not Only. By combining the two types of analysis for source codes with semantic analysis for written academic texts, the hybrid model allows the system to detect more profound emotional similarities and simple differences. The system aims to improve the detection of academic cheating, plagiarism, and code reuse by utilizing AI-driven semantic understanding. It also assists personnel in giving precise feedback. By ensuring that students are evaluated on their originality, comprehension, and coding skills rather than on unreliable comparisons, this system aims to increase the quality, fairness, and effectiveness of academic assessments.

In the digital age, educational institutions use automated systems to evaluate both written and programming tasks. However, traditional tools often struggle to grasp logical structures and deeper meanings. To solve this problem, our project employs a hybrid model specifically designed for academic evaluation. For source code, the system utilizes a Hybrid Stylometric + AST Structural Analysis.

Stylometric analysis: This determines a student's distinct coding style, including how they organize logic or name variables. It aids in separating AI patterns from human habits.

AST Structural Analysis: Even if the student tried to conceal it by changing names or comments, AST (Abstract Syntax Tree) Structural Analysis examines the code's skeleton to determine whether the underlying logic was produced by AI.

1.2 Purpose and Description

This thesis introduces a Semantic Text and Source Code AI Checker that combines semantic analysis of written texts with structural and functional analysis of programming code. The hybrid model can more accurately detect AI-assisted or plagiarized work by applying advanced algorithms to detect deeper logical and semantic patterns and straightforward similarities. The system provides staff with a helpful tool to enhance academic assessment, stop the misuse of AI tools, and provide students with constructive feedback to improve the quality of the evaluation process, fairness, and reliability.

Additionally, it promotes academic Integrity by ensuring student work accurately reflects their knowledge, abilities, and comprehension. Maintaining academic integrity by correctly differentiating between original student work and AI-generated code is the main objective of the Hybrid Stylometric + AST Structural Analysis. By offering a trustworthy detection technique that finds deep-level logic patterns that conventional checkers frequently overlook, it seeks to lessen students' reliance on AI tools.

This thesis presents a Semantic Text and Source Code AI Checker that combines structural and functional programming code analysis with written text semantic analysis. The hybrid model uses advanced algorithms to identify deeper logical and semantic patterns in addition to simple similarities, making it possible to locate AI-assisted or plagiarized work with greater accuracy. To improve the evaluation process's quality, fairness, and dependability, the system gives personnel a valuable tool to enhance academic assessment, prevent the abuse of AI tools, and give students constructive criticism. This hybrid algorithm evaluates source code submissions by combining two advanced methods. To distinguish human-written code from predictable AI patterns, Stylometric Analysis first determines a student's distinct "coding fingerprint" such as their particular habits for naming variables and structuring logic. Second, the underlying "skeleton" or logical tree of the code is examined by AST (Abstract Syntax Tree) Structural Analysis. Even if a student tries to hide AI-generated logic by renaming variables or altering comments, the algorithm can identify it by concentrating on the structural hierarchy rather than just the surface text. When combined,



these techniques offer a strong, multi-layered defense against academic dishonesty in programming assignments.

1.3 Objective of the Study

This study aims to detect AI-Generated semantic text and source code using a hybrid model algorithm.

Specifically, it aims to:

1. Develop a hybrid algorithm capable of processing data on AI-Generated content by analyzing semantic text and source code.

2. Design a hybrid model using RoBERTa, Hybrid Stylometric + AST Structural Analysis to distinguish AI-Generated semantic text and source code.

3. Implement the hybrid model algorithm and compare its performance and detection accuracy with approaches presented in previous studies



1.4 Scope and Limitations

The primary goal of this project is to create an AI checker for semantic text and source code using a hybrid model for academic evaluation. The system design helps the personnel ensure their students do not rely on AI. The system aims to analyze written texts and programming codes by combining machine learning (ML), natural language processing (NLP), and code similarity detection algorithms. Its scope includes identifying plagiarism or code reuse in source code submissions and detecting essays, sentences, and phrases produced by artificial intelligence. The system is intended to assist teachers in ensuring academic Integrity, keeping an eye on originality, and administering more accurate and equitable assessments.

The system can Only analyze programming codes and academic texts concerning academic submissions. It might not go into great detail about every programming language;

Instead, it will start with those usually used in classrooms, like Python, Java, and C/C++. Similarly, the quality of training datasets and detection models decides how accurately semantic analysis can identify AI-Generated and plagiarized content in the output of the students. While the system is a helpful tool for personnel, it does not entirely prevent academic dishonesty. External factors like the size of the dataset, the Limited computing resources, and the ongoing evolution of AI-Generated content may also impact the detection performance. Also, this system assures personnel that their students do not rely on AI.

1.5. Definition of Terms

Artificial Intelligence (AI) allows machines to perform operations like text production, decision-making, and problem-solving, which generally call for human's intelligence.

AI-Generated Content Texts or computer programs automatically created by AI tools (like ChatGPT and Copilot) without direct humans' authorship.

Academic assessment is evaluating student work, such as reports, essays, and programming assignments, to gauge performance, originality, and comprehension.

A hybrid model combines machine learning (ML), natural language processing (NLP), and code similarity detection methods to examine source code structural/functional patterns and textual semantic meaning.

Machine learning (ML) entails teaching algorithms to identify patterns and gradually increase detection accuracy without explicit programming.

Natural language Processing (NLP) an AI method for analyzing, interpreting, and comprehending humans language in a way that makes sense for computational processing is called natural language processing.

Plagiarism is the deliberate or inadvertent act of reproducing or presenting someone else's ideas, work, or code without giving due credit.

Semantic analysis is deciphering the context and meaning of words, phrases, and sentences in a text to identify conceptual or logical parallels that go beyond simple correspondences.

Source code is the human-readable set of instructions written in Programming language that tells a computer how to perform specific tasks or operations.

Source Code Detection uses algorithms and machine learning techniques to identify and analyze programming code to determine similarities, originality, and possible plagiarism.

Semantic text refers to the meaning and context of words or sentences, focusing on what the text conveys rather than its exact wording**.**

REVIEW OF RELATED LITERATURE



This study presents a hybrid model for academic assessment of an AI checker for semantic text and source code. The system is made to analyze programming code and written text more accurately. Deep semantic meaning and logical structures often get lost in traditional work methods. The suggested hybrid model integrates several strategies for improved detection and assessment to overcome this problem. The purpose of this tool is to assist personnel in recognizing academic fairness and originality. All stuff considered, it offers a more reliable way to evaluate student work in the digital era.

The use of semantic similarity tools in automated content scoring of fact-based essays written by EFL learners

This study searched for open-source semantic similarity tools and evaluated their effectiveness in automated content scoring of fact-based essays written by English-as-a-Foreign-Language (EFL) learners. Fifty writing samples were collected under a fact-based writing task from an academic English course in a Japanese university, and a native expert produced a gold standard. A shortlist of carefully selected tools, including InferSent, spaCy, DKPro, ADW, Similarity, and Latent Semantic Analysis, Generated semantic similarity scores between student writing and expert samples. [1]

Towards a Humans-AI Hybrid System for Categorizing Programming Problems

As programming skills are increasingly required worldwide and across disciplines, many students use online platforms that provide automatic feedback through a Programming Online

Judge (POJ) mechanism. POJs are very popular e-learning tools, boasting extensive collections of programming problems. Despite their many benefits, students often struggle when solving incompatible issues with their prior knowledge. [2]

Plagiarism Checker

Plagiarism is stealing another individual's intellectual property, comprising their literary work, creative output, or ideas. Claiming these as one's own without due acknowledgment and consent directly violates the principles of academic Integrity, honesty, and originality. [3]

Detecting AI-Generated Code Assignments Using Perplexity of Large Language Models

Large language models like ChatGPT can generate humans-like code, posing challenges for programming education as students may be tempted to misuse them on assignments. However, no robust detectors are explicitly designed to identify AI-Generated code. According to [4]

Investigating Code Generation Performance of ChatGPT with Crowdsourced Social Data

The recent advancements in Artificial Intelligence, particularly in large language models and generative models, are reshaping the field of software engineering by enabling innovative ways of performing various tasks, such as programming, debugging, and testing. However, few existing works have thoroughly explored the potential of AI in code generation and users' attitudes toward AI-assisted coding tools. This knowledge gap makes it unclear how AI transforms software engineering and programming education.[5]

Expert-Authored and Machine-Generated Short-Answer Questions for Assessing Students' Learning Performance on JSTOR

Humans-guided machine learning can improve computing intelligence and accurately assist humans in various tasks. In education research, artificial intelligence (AI) is applicable in many situations, such as predicting students' learning paths and strategies. In this study, we explore how repetitive practice of short-answer questions can enhance students' long-term memory for subsequent improvements in learning performance. However, frequent authoring questions and grading require teachers' professionalism, experience, and considerable effort. [6]



An AI-Based System for Formative and Summative Assessment in Data Science Courses

Massive open online courses (MOOCs) provide hundreds of students with teaching materials, assessment tools, and collaborative instruments. The assessment activity, in particular, is demanding in terms of time and effort; thus, artificial intelligence can help address and reduce the time and effort required. [7]

Automated Test Generation and Marking Using LLMs

This paper presents an innovative exam-creation and grading system powered by advanced natural language processing and local large language models. The system automatically generates straightforward, grammatically accurate questions from short passages and longer documents across different languages, supports multiple formats and difficulty levels, and ensures semantic diversity while minimizing redundancy, thus maximizing the percentage of the material covered in the Generated exam paper. [8]

Semantic Text Classification for Supporting Automated Compliance Checking in Construction

Automated regulatory and contractual compliance checking requires automated rule extraction from regulatory and contractual textual documents (e.g., contract specifications). Automated rule extraction is a challenging task that requires complex text processing. In the proposed automated compliance checking (ACC) approach, the first step in automating the rule extraction process is automatically classifying the different documents and parts of documents

(e.g., contract clauses) into predefined categories (environmental, safety, health, etc.) for preparing them for further text analysis and rule extraction.[9]

Automated functional dependency detection between test cases using Doc2Vec and Clustering

Knowing about dependencies and similarities between test cases is beneficial for prioritizing them for cost-effective test execution. This holds especially true for the time-Consuming, manual execution of integration test cases written in natural language. Test case dependencies are typically derived from requirements and design artifacts. However, such artifacts are not always available, and the derivation process can be time Consuming. In this paper, we propose, apply, and evaluate a novel approach that derives test cases' similarities and functional dependencies directly from the test specification documents written in natural language, without requiring any other data source.[10]

Vulnerability Detection of Ethereum Smart Contract Based on SolBERT-BiGRU-Attention Hybrid Neural Model

In recent years, with the great success of pre-trained language models, the pre-trained BERT model has been gradually applied to source code understanding. However, the time cost of training a language model from scratch is very high, and how to transfer the pre-trained language model to the field of intelligent contract vulnerability detection is a hot research direction. This paper proposes a hybrid model to detect common vulnerabilities in smart contracts based on a lightweight pre-trained language model, BERT, connected to a bidirectional gate recurrent unit model. [11]

Designing an AI-Supported Framework for Literary Text Adaptation in Primary Classrooms

The proposed system enables educators to simplify age-specific text, visual re-narration, reinvention, and multilingual augmentation through modular tools. Central to the design is the Ethical–Pedagogical Validation Layer (EPVL), a GPT-powered auditing module that evaluates AI-Generated content across four normative dimensions: developmental appropriateness, cultural sensitivity, semantic fidelity, and ethical transparency[12]

Plagiarism types and detection methods: a systematic survey of algorithms in text analysis

Plagiarism in academic and creative writing continues to be a significant challenge, driven by the exponential growth of digital content. This paper presents a systematic survey of various types of plagiarism and the detection algorithms employed in text analysis. We categorize plagiarism into distinct types, including verbatim, paraphrasing, translation, and idea-based plagiarism, discussing the nuances that make detection complex.[13]

Integrated Technique of Natural Language Texts and Source Codes Authorship Verification in The Academic Environment

The issue of text plagiarism in academic and educational environments is becoming increasingly relevant every year. The quality of research articles and works is declining due to students copying fragments of others' works and using modern generative models for text and source code creation. The article proposes an integrated technique for authorship verification of natural and programming language texts, based on statistical methods, machine learning, and deep neural networks. The presented technique addresses several related tasks: assessing text homogeneity, detecting plagiarism when solving closed-set authorship attribution problems, and identifying texts and fragments created by generative models. [14]

A Hybrid Approach for Automatic Question Generation from Program Codes. | EBSCOhost

Generating questions is one of the most challenging tasks in the natural language processing discipline. With the significant emergence of electronic educational platforms like e-learning systems and the extensive scalability of e-learning, there is an increased urge to generate intelligent and deliberate questions to measure students' understanding. Many works have been done in this field using different techniques; however, most approaches work on extracting questions from text. [15]

Survey on AI-Generated Plagiarism Detection: The Impact of Large Language Models on Academic Integrity

A survey conducted in 2023 surveyed 3,017 high school and college students. It found that almost one-third of them confessed to using ChatGPT for assistance with their homework. The rise of Large Language Models (LLMs) such as ChatGPT and Gemini has led to a surge in academic misconduct. Students can now complete their assignments and exams just by asking an LLM for solutions to the given problem, without putting in the effort required for learning. What is more worrying as educators lack the proper tools to detect it. The more advanced AI tools become, the more humans-like the text it generates, and the more difficult it are to detect. [16]

An Automated Hybrid Exam Evaluation Framework for Textual Courses Using AI

Evaluation and assessment systems stand to benefit from Artificial Intelligence (AI) driven, automated, open-ended exam grading technologies. However, despite remarkable achievements in Natural Language Processing (NLP) research, streamlining of those achievements into production-quality systems has yet to be seen. In this paper, we present the specification, design, and a prototype implementation of an open, extensible, and comprehensive software platform and illustrate its performance via the results of our initial experimentation with real data obtained in collaboration with an operating college in Turkey. [17]

Academic Plagiarism Detection

This article summarizes the research on computational methods to detect academic plagiarism by systematically reviewing 239 research papers published between 2013 and 2018. To structure the presentation of the research contributions, we propose novel technically oriented typologies for plagiarism prevention and detection efforts, the forms of academic plagiarism, and computational plagiarism detection methods. We show that academic plagiarism detection is a highly active research field. [18]

Shaping the Future of Higher Education: A Technology Usage Study on Generative AI Innovations

Generative Artificial Intelligence (GAI) is rapidly reshaping the landscape of higher education, offering innovative solutions to enhance student engagement, personalize learning experiences, and improve academic performance prediction. This study explores GAI applications in educational contexts, drawing insights from 67 case studies meticulously selected from over 300 papers presented at the AIED 2024 conference. The research focuses on eight key themes, from student engagement and behavior analysis to integrating generative models into educational tools. [19]



Detection of artificial intelligence-Generated essays for academic assessment Integrity using large language models

Across various fields of humans life, the lightning adoption of generative artificial intelligence (AI) tools is driven by their ease of use and ability to produce high-quality outputs, transforming communication and productivity. However, the growing popularity and influence of models such as ChatGPT have raised concerns in education regarding academic Integrity, as AI-Generated content challenges the authenticity of student work and complicates traditional assessment practices. Existing plagiarism detection tools, such as Turnitin and GPTZero, attempt to identify AI-Generated content; however, their reliability remains Limited, as most struggle to accurately differentiate between humans and AI essays. [20]

Detecting AI-assisted submissions in introductory programming via code anomaly

Artificial Intelligence (AI) can foster education but can be misused to breach academic Integrity. Large language models like ChatGPT can generate solutions for individual assessments that are expected to be completed independently. There are several automated detectors for AI-assisted work. However, most are not dedicated to programming, and/or rely on existing student submissions (i.e., the learning approach). This paper presents a straightforward detector for AI-assisted code, relying on code anomalies. No existing student submissions are needed. [21]



A systematic literature review on source code similarity measurement and clone detection: Techniques, applications, and challenges

Measuring and evaluating source code similarity is a fundamental software engineering activity that embraces a broad range of applications, including but not Limited to code recommendation, duplicate code, plagiarism, malware, and smell detection. This paper proposes a systematic literature review and meta-analysis on code similarity measurement and evaluation techniques to shed light on the existing approaches and their characteristics in different applications. We initially found over 10,000 articles by querying four digital libraries and ended up with 136 primary studies. [22]

Deep learning detection method for large language models-Generated scientific content

Large Language Models (LLMs), such as GPT-3 and BERT, reshape how textual content is written and communicated. These models can generate indistinguishable scientific content from humans-authored work, raising concerns within the scientific community. Mitigating the risk of LLM-facilitated research fabrication and disseminating falsified data and results requires robust safeguards to maintain the Integrity of scientific publications. This research paper presents a novel ChatGPT-Generated scientific text detection method, AI-Catcher. [23]

Detection of Artificial Intelligence-Generated Essays for Academic Assessment Integrity using Large Language Models

Across various fields of human's life, the lightning adoption of generative artificial intelligence (AI) tools is driven by their ease of use and ability to produce high-quality outputs, transforming communication and productivity. However, the growing popularity and influence of models such as ChatGPT have raised concerns in education regarding academic Integrity, as AI-Generated content challenges the authenticity of student work and complicates traditional assessment practices. Existing plagiarism detection tools, such as Turnitin and GPTZero, attempt to identify AI-Generated content; however, their reliability remains Limited, as most struggle to accurately differentiate between humans and AI essays. To address existing tools' Limitations, we propose a novel detection model leveraging three large language models (LLMs): Generative Pre-trained Transformer 2 (GPT-2), Robustly Optimized BERT Pretraining Approach (RoBERTa), and Bidirectional and Auto-Regressive Transformers (BART). [24]

A Generative AI-Driven Method-Level Semantic Clone Detection Based on the Structural and Semantical Comparison of Methods

Code fragments with identical or similar functionality are called code clones. This study aims to detect semantic clones in Java-based programs, focusing on the method-level granularity. To accomplish this, our approach extracts and compares both semantical and structural features of Java methods and applies specific heuristics to obtain the final result set. The semantics of a method are often described by its documentation, while its implementation within the code body characterizes its structural details. However, Java methods may not always be accompanied by descriptive documentation. [25]

Can Large Language Models Detect Plagiarism in Source Code?

The issue of code plagiarism represents a significant challenge in The Academic environment. This study examines the potential of large language models (LLMs) in improving code plagiarism detection. The performance of several LLMs, including GPT-4o, GPT-3.5 Turbo, LLaMA 3, and CodeLlama is evaluated in comparison to conventional tools, such as JPlag, across various levels of code plagiarism. The findings of our study illustrate that state-of-the-art LLMs can outperform traditional methods, particularly in detecting sophisticated forms of plagiarism. [26]

AI vs. Humans Writing: Developing a Novel Method for Text Authenticity Detection in Education

The most recent advancements in generative artificial intelligence (AI) technology have created significant problems distinguishing between AI-Generated and humans-produced text, especially in education. This issue raises concerns regarding submissions' academic Integrity and originality, especially in postsecondary education, where academic papers are a significant vehicle for assessing student learning. Research has shown that distinguishing between AI-Generated and Humans-written texts is a task in which humans' entities and AI detection technologies are arduous. For example, in an experiment conducted in AI authorship detection, humans were found to identify humans' authorship with 63% accuracy, but could Only identify

AI authorship with 24% accuracy. Another study involving senior editorial board members indicated their problematic version of distinguishing between humans and AI authorship, wherein accurate identification occurred Only 50% of the time. The Limitations of the current AI content detectors have been highlighted in various studies. Flitcroft et al. found examples of AI detectors misidentifying humans-Generated content as AI-Generated [27]

Semantic clone detection without type Limitations

Currently available tools for semantic clone detection have severe type Limitations related to objects, particularly instantiation and comparison. Recent research has focused on improving performance rather than removing these constraints. This paper proposes a test-based approach for semantic clone detection in Java that overcomes these problems. The test-based approach consists of running two methods with the same Input and comparing their outputs.[28]

Semantic Similarity Search for Source Code Plagiarism Detection: An Exploratory Study

Source code plagiarism detection (SCPD) is a crucial challenge in computer science education that affects academic Integrity. It can be considered as an Information Retrieval (IR) task. One of the IR approaches is the Semantic Similarity Search (S-3), which aims to retrieve

related results when given a query. It can be applied to SCPD by obtaining the most similar pairs given an extensive collection of codes.[29]

MixRevDetect: Towards Detecting AI-Generated Content in Hybrid Peer Reviews.

The growing use of large language models (LLMs) in academic peer review poses significant challenges, particularly distinguishing AI-Generated content from Humans-written feedback. This research addresses the problem of identifying AI-Generated peer review comments, which are crucial to maintaining the Integrity of scholarly evaluation. Prior research has primarily focused on generic AI-Generated text detection or on estimating the fraction of peer reviews that may be AI-Generated, often treating reviews as monolithic units. However, these methods fail to detect finer-grained AI-Generated points within mixed-authorship reviews. To address this gap, we propose MixRevDetect, a novel method to identify AI-Generated points in peer reviews. Our approach achieved an F1 score of 88.86%, significantly outperforming existing AI text detection methods.[30]

BM25-CTF: Improving TF and IDF factors in BM25 by using collection term frequencies

In this paper, the use of collection term frequencies (i.e., the total number of occurrences of a term in a document collection) in the BM25 retrieval model is investigated by modifying its term frequency (TF) and inverse document frequency (IDF) components. Using



selected examples extracted from TREC collections, it was observed that the informative nature, for retrieval purposes, of terms, either with the same TF (in a document) or IDF (in a collection), may be better revealed with the use of collection term frequencies (CTF). The collection term frequencies were integrated to TF and IDF factors from three new heuristics based on those observations and deviations from a random Poisson model. The novel formulations were tested by employing the TREC-1 to TREC-8 collections in the ad hoc task, for which BM25 was first developed and tested. Consistent and significant improvements were observed in mean average precision (MAP), reaching 17.67% for the TREC-8 dataset and 7.16% for all tested collections. These results were considerably better compared to other approaches surveyed aiming to improve BM25, proving the effectiveness of the proposed heuristics and formulae. The proposed approach requires Only additional offline pre-computations and does not entail extra computational complexity for retrieval while keeping the original spirit and parameter robustness of BM25.[31]

Analyzing the Performance of Sentiment Analysis Using BERT, DistilBERT, RoBERTa, and GPT-2

Sentiment analysis, also known as opinion mining, employs natural language processing (NLP) techniques to determine, extract, and measure the emotional undertones within a text. This approach is valuable for measuring public sentiment and interests across various domains, including social media discussions and product and movie reviews. Techniques like Linear Regression, Support Vector Machines, Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and Long Short-Term Memory (LSTM) are utilized in analyzing sentiments.



However, these are not comparable to the capabilities of the pre-trained transformer models such as BERT, DistilBERT, RoBERTa, and the recent large language model (LLM) GPT-2, which offer significantly enhanced performance and flexibility.[32]

Malicious Code Detection under 5G HetNets Based on a Multi-Objective RBM Model | IEEE Journals & Magazine | IEEE Xplore

The fifth generation (5G) mobile communication technology brings people a higher perceived rate experience, high-quality service, high-density user connection, and other commercial applications. As an essential means of data processing in 5G heterogeneous networks (HetNets), data fusion technology faces many malicious code attacks. Thus, it is imperative to find an efficient malicious code detection method. However, in the traditional research, due to dataset imbalance, the complexity of the deep learning network model, using a single-objective algorithm, and other factors, it brings greater loss and lower detection accuracy. Therefore, choosing a suitable network model and improving the data classification accuracy in HetNets is a significant challenge.[33]

Improvements to BM25 and language models examined

Recent work on search engine ranking functions reports improvements on BM25 and Language Models with Dirichlet Smoothing. In this investigation, nine recent ranking functions (BM25, BM25+, BM25T, BM25-adpt, BM25L, TF1°δ°p×ID, LM-DS, LM-PYP, and LM-PYP-TFIDF) are compared by training on the INEX 2009 Wikipedia collection and testing on INEX 2010 and 9 TREC collections. We find that once trained (using particle swarm optimization), there is very



little difference in performance between these functions, relevance feedback is effective, stemming is effective, and it remains unclear which function is best overall.[34]

Using a cosine similarity classifier for NLP analysis in the construction field texts

The paper considers some authors' approaches to developing a machine translation system to handle texts specialized in the construction industry in Russian and English. Graph theory is the basis for automatic sentence analysis and the representation of its structure. At the first stage, the sentence is represented by a line graph. The article discusses how to assign attributes to each word according to its prospective belonging to different parts of speech for their subsequent processing using graph convolutional neural networks (GCN). The initial features of each element of the sentence are compared with the calculated values obtained. By means of specific parameters for the compiled vectors, it are classified as parts of speech. The paper contains parts of the software listing for the computer implementation of the proposed technology.[35]

Enhancing academic integrity among students in GenAI Era:A holistic framework

Public software repositories such as GitHub make transparent the development history of an open-source software system. Source code commits, discussions about new features and bugs, and code reviews are stored and carefully attributed to the appropriate developers. However, sometimes governments may seek to analyze these repositories, to identify citizens who contribute to projects they disapprove of, such as those involving cryptography or social media. While developers who seek anonymity may contribute under assumed identities, their



body of public work may be characteristic enough to betray who they really are. The ability to contribute anonymously to public bodies of knowledge is extremely important to the future of technological and intellectual freedoms. Just as in security hacking, the only way to protect vulnerable individuals is by demonstrating the means and strength of available attacks so that those concerned may know of the need and develop the means to protect themselves.[36]

Analysis and Detection of a Radical Extremist Discourse Using Stylometric Tools

The amount of radical extremist texts published online is constantly growing with researchers as well as agencies working hard to design tools to detect and remove this type of content. However, a great deal of this research effort is being focused on English-language materials. Meanwhile, in the last few years extremist groups including radical Islamic ones have started to produce content in languages other than English and, more specifically, in Russian which has been increasingly employed. Qualitative analysis of this content is important but not sufficient. We show the possibility of using stylometric tools allowing different types of texts analysis to be performed in order to analyze and detect radical extremist content. Using a meticulously designed dataset consisting of real-world extremist forum texts on two topics and texts by common Internet users on the same topics, we were able to show that the authors from extremist forum remain consistent in their style irrespective of the topic and their texts could thus be detected using a variety of methods. We conclude by the underscoring the importance of combining qualitative and quantitative analysis of a radical extremist discourse for better understanding of radical minds and development of counter-extremist tools.[37]

Multimodal stylometry: A novel approach for authorship identification.

This dissertation introduces multimodal stylometry, a novel approach to authorship identification that integrates text and source code features for a comprehensive understanding of an author's unique style. Traditional stylometric methods have primarily focused on either text stylometry or source code stylometry, thereby neglecting the potential insights that multimodality may provide. This research aims to bridge this gap by proposing a framework that combines textual and source code data to enhance the accuracy and reliability of authorship identification. The study begins by reviewing existing literature on authorship identification and stylometry, highlighting the limitations of unimodal approaches.[38]

Stylometry for real-world expert coders: a zero-shot approach

Code stylometry is the application of stylometry techniques to determine the authorship of software source code snippets. It is used in the industry to address use cases like plagiarism detection, code audits, and code review assignments. Most works in the code stylometry literature use machine learning techniques and (1) rely on datasets coming from in vitro coding competition for training, and (2) only attempt to recognize authors present in the training dataset (in-distribution authors). In this work we give a fresh look at code stylometry and challenge both these assumptions: (1) we recognize expert authors who contribute to real-world open-source projects, and (2) we show how to accurately recognize authors not present in the training set (out-distribution authors). We assemble a novel open dataset of code snippets for code stylometry tasks consisting of 114,400 code snippets, authored by 104 authors having contributed 1,100 snippets each. We develop a K-nearest neighbors algorithm (k-NN) classifier

for the code stylometry task and train it on the dataset. Our system achieves a top accuracy of 69% among five randomly selected in-distribution authors, thus improving state of the art by more than 20%. We also show that when moving from in-distribution to out-distribution authors, the classification performances of the k-NN classifier remain the same, achieving a top accuracy of 71% among five randomly-selected out-distribution authors.[39]

A Complex Network Approach to Stylometry

Statistical methods have been widely employed to study the fundamental properties of language. In recent years, methods from complex and dynamical systems proved useful to create several language models. Despite the large amount of studies devoted to represent texts with physical models, only a limited number of studies have shown how the properties of the underlying physical systems can be employed to improve the performance of natural language processing tasks. In this paper, I address this problem by devising complex networks methods that are able to improve the performance of current statistical methods. Using a fuzzy classification strategy, I show that the topological properties extracted from texts complement the traditional textual description. In several cases, the performance obtained with hybrid approaches outperformed the results obtained when only traditional or networked methods were used. Because the proposed model is generic, the framework devised here could be straightforwardly used to study similar textual applications where the topology plays a pivotal role in the description of the interacting agents.[40]

Multimodal stylometry: A novel approach for authorship identification.

This dissertation introduces multimodal stylometry, a novel approach to authorship identification that integrates text and source code features for a comprehensive understanding of an author's unique style. Traditional stylometric methods have primarily focused on either text stylometry or source code stylometry, thereby neglecting the potential insights that multimodality may provide. This research aims to bridge this gap by proposing a framework that combines textual and source code data to enhance the accuracy and reliability of authorship identification. The study begins by reviewing existing literature on authorship identification and stylometry, highlighting the limitations of unimodal approaches. Leveraging recent advancements in multimodal biometrics and feature fusion, the research introduces a methodology that extracts stylometric features from written text and source code. These multimodal features are then integrated using an extended feature fusion technique that introduces an extra layer of feature selection. To validate the proposed approach, a diverse dataset comprising texts and corresponding source code data from various authors is curated. The dissertation explores the effectiveness of multimodality when compared to unimodality. Furthermore, the research investigates the transferability of the proposed multimodal stylometry framework in distinguishing AI and Human generated text and source code. The findings not only advance authorship identification techniques but also hold implications for applications in forensic linguistics, digital humanities, and content analysis. Ultimately, this research underscores the significance of multimodal stylometry in estimating the identity of an author.[41]

AST: Analyzing and Modifying Java ASTs with Python

Analyzing and modifying source code at the Abstract Syntax Tree (AST) level is fundamental to numerous software engineering tasks, including program analysis, instrumentation, and transformation. While Java-specific tools exist, they often operate at the bytecode level or are tightly coupled to the Java ecosystem, limiting their flexibility and accessibility. In this paper, we present jAST, a Python-based tool that generates and manipulates ASTs for Java programs. By leveraging Python's simplicity and extensive ecosystem, jAST enables precise source-level analysis, seamless integration with Python workflows, and support for advanced tasks such as feature extraction for debugging and learning-based failure analysis. jAST

is an open-source tool offering researchers, educators, and practitioners an extensible framework for working with Java ASTs.[42]

Development of a system for detecting refactorings in software code history

Refactoring improves a codebase's structure without changing its behavior, helping both researchers and practitioners maintain large systems. RefactoringMiner is a strong detector for Java projects, but it is limited to a single language. In this thesis, a language-agnostic core and an adapter are introduced. As a first case study, Python support is added. An ANTLR4 parser produces a parse tree that is traversed to build an AST structure based on JDT entities; an adapter then maps that AST to RefactoringMiner's existing UMLModel so the unchanged detection engine can run unmodified. The prototype was evaluated on ~380 unit tests covering ~38 refactoring types and illustrates its behavior on ten open-source Python histories. Because there is no established Python refactoring benchmark with ground-truth labels, the open-source runs are treated as qualitative case studies and do not report precision/recall for them; instead, representative detections and failure modes are presented. On the unit suite, the tool attains 94.7% micro-averaged recall. The contributions are: (i) a language-neutral architecture for RefactoringMiner, (ii) a Python front end consisting of an ANTLR4, AST structure, UMLModel adapter pipeline, and (iii) an evaluation that surfaces both strengths and limits, outlining a path to multi-language refactoring detection.[43]

An Extensive Study of the Structure Features in Transformer-based Code Semantic Summarization

Transformers are now widely utilized in code intelligence tasks. To better fit highly structured source code, various structure information is passed into Transformer, such as positional encoding and abstract syntax tree (AST) based structures. However, it is still not clear how these structural features affect code intelligence tasks, such as code summarization. Addressing this problem is of vital importance for designing Transformer-based code models. Existing works are keen to introduce various structural information into Transformers while lacking persuasive analysis to reveal their contributions and interaction effects. In this paper, we conduct an empirical study of frequently-used code structure features for code representation, including two types of position encoding features and AST-based structure features. We propose a couple of probing tasks to detect how these structure features perform in Transformer and conduct comprehensive ablation studies to investigate how these structural features affect code semantic summarization tasks. To further validate the effectiveness of code structure features in code summarization tasks, we assess Transformer models equipped with these code structure features on a structural dependent summarization dataset. Our experimental results reveal several findings that may inspire future study: (1) there is a conflict between the influence of the absolute positional embeddings and relative positional embeddings in Transformer; (2) AST-based code structure features and relative position encoding features show a strong correlation and much contribution overlap for code semantic summarization tasks indeed exists between them; (3) Transformer models still have space for further improvement in explicitly understanding code structure information.[44]

Static Analysis by Abstract Interpretation Collecting Types of Python Programs

Software bugs are costly: they can have disastrous consequences on critical systems, but also on more common applications such as Web servers. It is thus interesting to possess tools helping developers detect those bugs before they ship software into production. The most common approach to avoid those errors is to use testing, which is inefficient: tests are usually written by hand, taking time, and they cover a few executions of a program, but not all of them. The approach I undertook during this internship is to design a static analyzer by abstract interpretation for Python. Static analyzers are programs that automatically analyze an input program and report errors this input program may contain (given a certain class of errors the static analyzer may detect). In particular, if a static analyzer is sound and reports no error on a given input, we know that this program does not contain any error from the class the static analyzer is able to discover. Abstract interpretation is a theory formalizing links between a precise but costly semantics with some computable, approximated counterpart, thus guiding the design and implementation of sound static analyzers. State of the art static analyzers include Julia (analyzing Java), Astrée and Sparrow (for C software). Those are only analyzing statically typed languages. There are no mature static analyzers available for dynamic languages such as JavaScript and Python, although a few static analyses have already been developed and implemented. During this internship, I developed a static analysis of Python programs collecting the types of each variable, and possible type error exceptions. Python is a popular programming language, especially used in teaching and in the scientific community, well-known for its powerful syntax. Major software projects written in Python include the Django web framework and the SageMath computer algebra system. Python is an object-oriented dynamic language,

where every value is an object. Python's dynamism means that undeclared variables, type incompatibilities, \dots\, are exceptions detected at runtime.[45]

PARSI: Persian Authorship Recognition via Stylometric Integration

The intricate linguistic, stylistic, and metrical aspects of Persian classical poetry pose a challenge for computational authorship attribution. In this work, we present a versatile framework to determine authorship among 67 prominent poets. We employ a multi-input neural framework consisting of a transformer-based language encoder complemented by features addressing the semantic, stylometric, and metrical dimensions of Persian poetry. Our feature set encompasses 100-dimensional Word2Vec embeddings, seven stylometric measures, and categorical encodings of poetic form and meter. We compiled a vast corpus of 647,653 verses of the Ganjoor digital collection, validating the data through strict preprocessing and author verification while preserving poem-level splitting to prevent overlap. This work employs verse-level classification and majority and weighted voting schemes in evaluation, revealing that weighted voting yields 71% accuracy. We further investigate threshold-based decision filtering,

allowing the model to generate highly confident predictions, achieving 97% accuracy at a 0.9 threshold, though at lower coverage. Our work focuses on the integration of deep representational forms with domain-specific features for improved authorship attribution. The results illustrate the potential of our approach for automated classification and the contribution to stylistic analysis, authorship disputes, and general computational literature research. This research will facilitate further research on multilingual author attribution, style shift, and generative modeling of Persian poetry.[46]

CHAPTER 3

TECHNICAL BACKGROUND

3.1 System Architecture

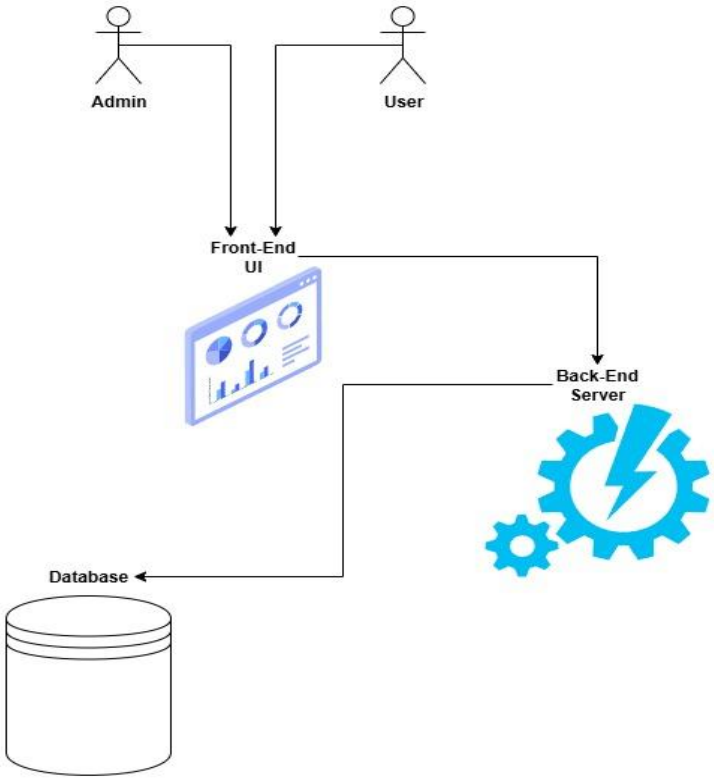


Figure 1. System Architecture

Admin and User interact with the system.

Front-End UI:



The web interface where users interact.

Sends and receives data from the Back-End Server.

Back-End Server:

Handles business logic and processes user requests.

Communicates with the Database.

Database:

Stores all application data.

Only accessed by the Back-End Server.



1.2 Conceptual Diagram

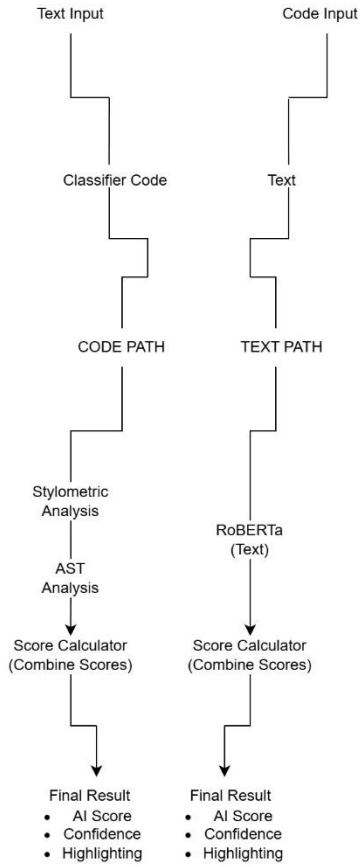

Text Input
Code Input
Classifier Code
Text
CODE PATH
TEXT PATH
Stylometric Analysis
AST Analysis
RoBERTa (Text)
Score Calculator (Combine Scores)
Score Calculator (Combine Scores)
Final Result
AI Score
Confidence
Highlighting
Final Result
AI Score
Confidence
Highlighting

Figure 2. Conceptual Diagram

1. **User Flow:**

- The User opens The TruthScan Al website (Landing Page).
- The User uploads text or code to check If it's AI-Generated.
- The Front-End UI sends the text to the Back-End Server.
- The Back-End sends it to the AI Detection Engine for checking.
- The Al Engine analyzes the text and returns the result/score.
- The Back-End saves the result to the Database.
- The Front-End shows the detection result to the User.

2. **Admin Flow:**

- The Admin logs in using the login page.
- The Front-End UI sends the credentials to the Back-End Server.
- The Server checks the credentials in the Database. Once verified, the Admin can access the Dashboard.
- From the Dashboard, the Admin can view user activity, monitor system data, and change system Settings.
- Any updates or Settings are sent through the Back-End and saved in the Database.

3. **User Flow:**

- The User opens The TruthScan Al website (Landing Page).

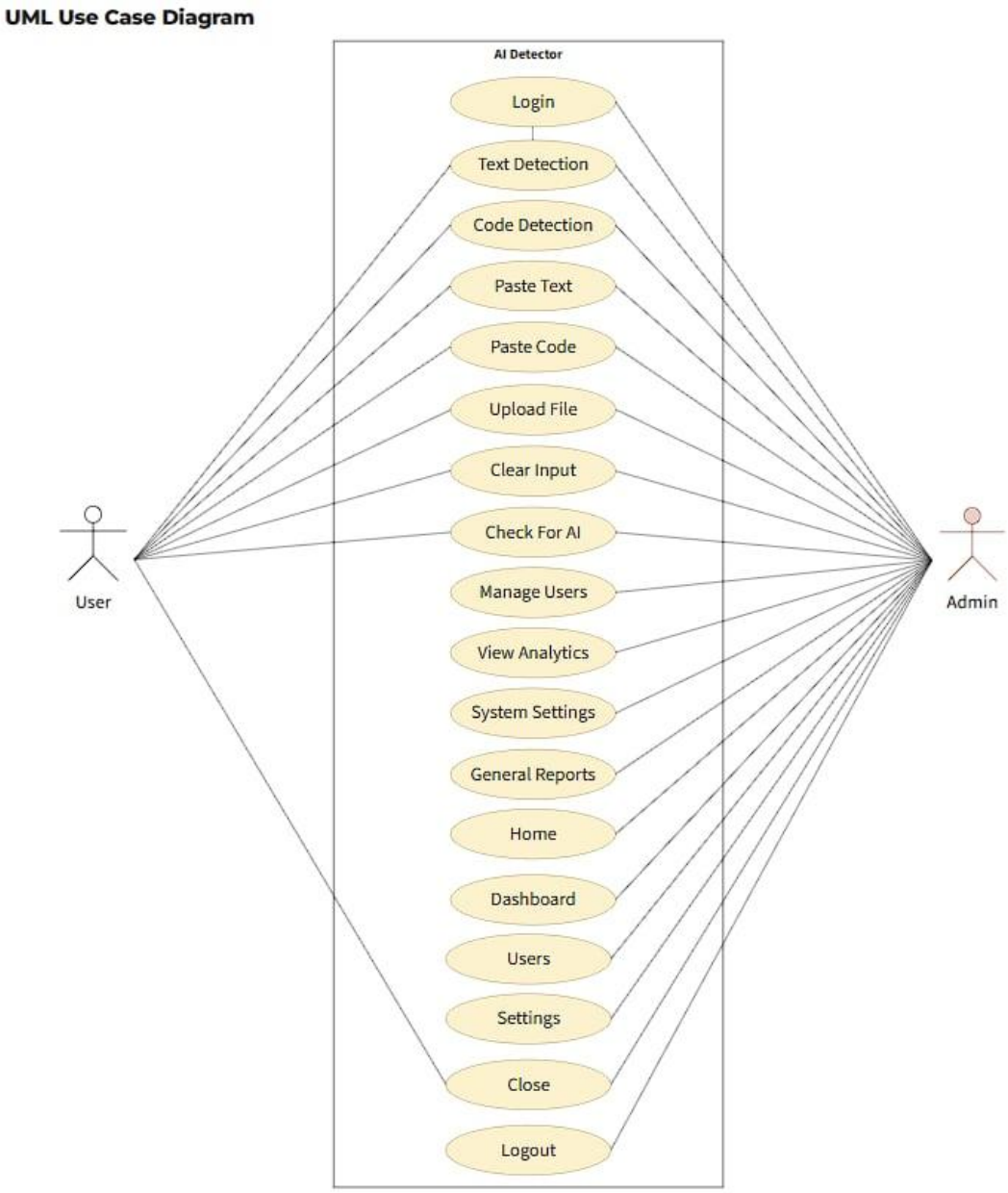


- The User uploads text or code to check If it's AI-Generated.
- The Front-End UI sends the text to the Back-End Server.
- The Back-End sends it to the AI Detection Engine for checking.
- The Al Engine analyzes the text and returns the result/score.
- The Back-End saves the result to the Database.
- The Front-End shows the detection result to the User.

4. Admin Flow:

- The Admin logs in using the login page.
- The Front-End UI sends the credentials to the Back-End Server.
- The Server checks the credentials in the Database. Once verified, the Admin can access the Dashboard.
- From the Dashboard, the Admin can view user activity, monitor system data, and change system Settings.
- Any updates or Settings are sent through the Back-End and saved in the Database.





1.3 Use Case Diagram

Figure 3. Use Case Diagram

Key Components of the Use Case Diagram

1. Actors
    - User – interacts with the system mainly for content submission and AI detection.
    - Admin – has complete control, including monitoring and managing system operations, and can also interact with the system as the user.
2. System
    - AI Checker – represents the system boundary containing all use cases accessible to users and admins.

3. Use Cases
    - Login / Logout – for secure access and exit exclusive for admin.
    - Paste Text / Upload File / Straightforward Input / Check for AI – core content Input and analysis functions.
    - Home / Dashboard / User / Settings – navigation and personalization features.
    - Manage Users / View Analytics / System Settings / Generate Reports – administrative functions for monitoring and maintaining the system.

1.4 Sequence Diagram

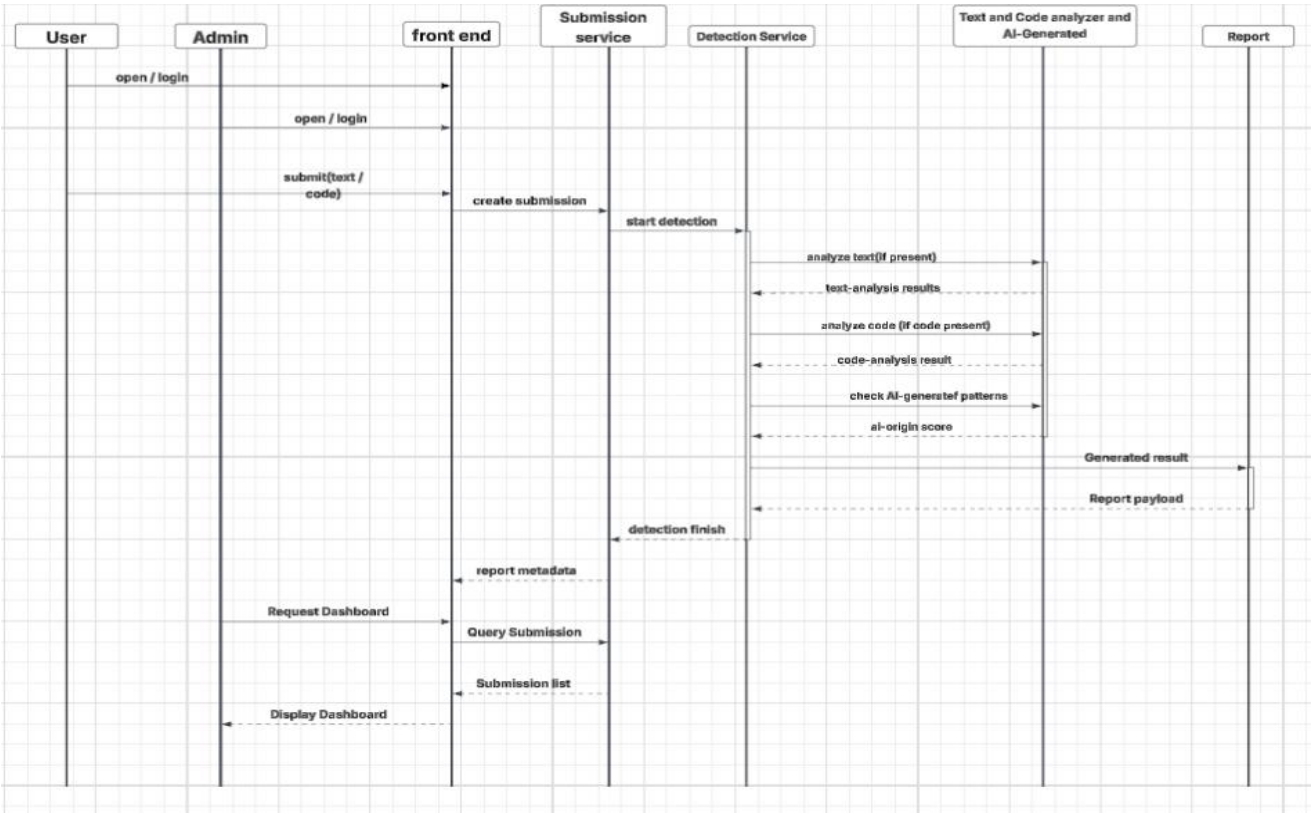


Figure 4. Sequence Diagram

The figure illustrates the dynamic interaction between the Semantic Text and Source Code AI checker system components using a Hybrid Model Algorithm. It demonstrates the sequential process flow, from user interaction to the final generation of detection reports.

1.5 ERD (Database Design)

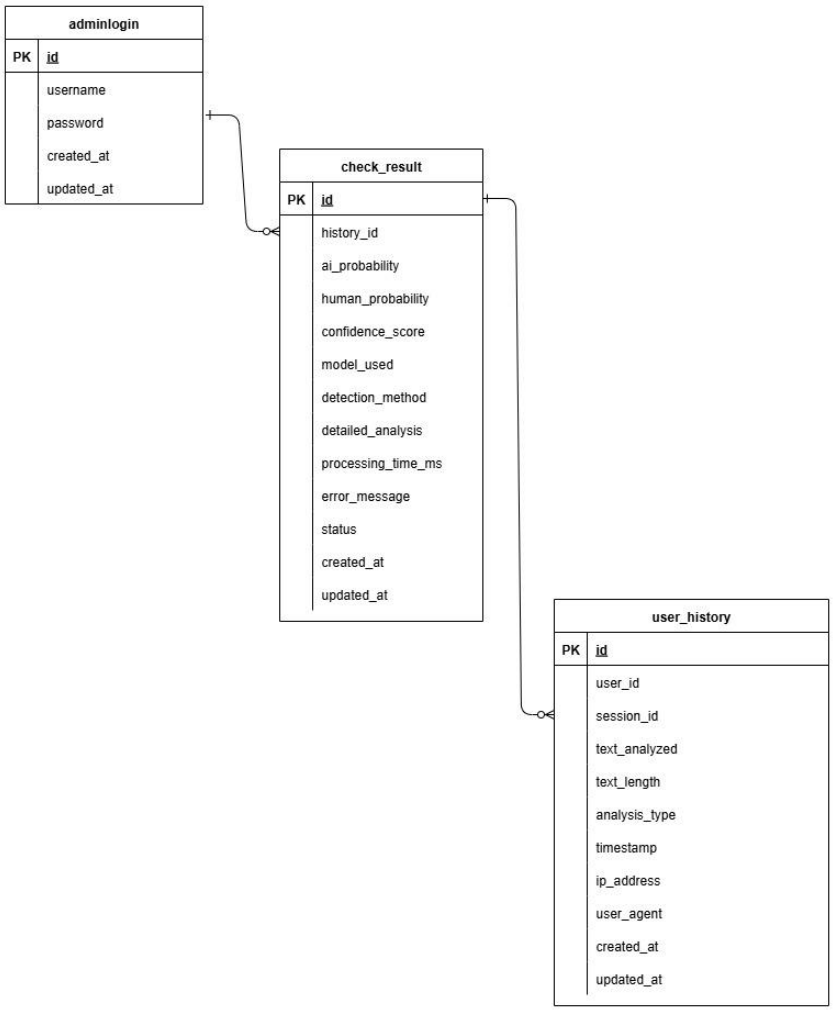


Figure 5. ERD (Database Design)

The Admin table is the primary storage for administrator account information in the system database. It contains essential fields that support authentication, account management, and

activity tracking. The ID field is the primary key (PK), uniquely identifying each admin record. The username and password fields store the administrator's login credentials, which are required for secure system access. The created_at field records the date and time the account was created, while the updated_at field logs the most recent changes to the account information. This structure ensures secure and organized management of administrator data, enabling reliable authentication and easy maintenance.

3.6. User Interface Design (Prototype)

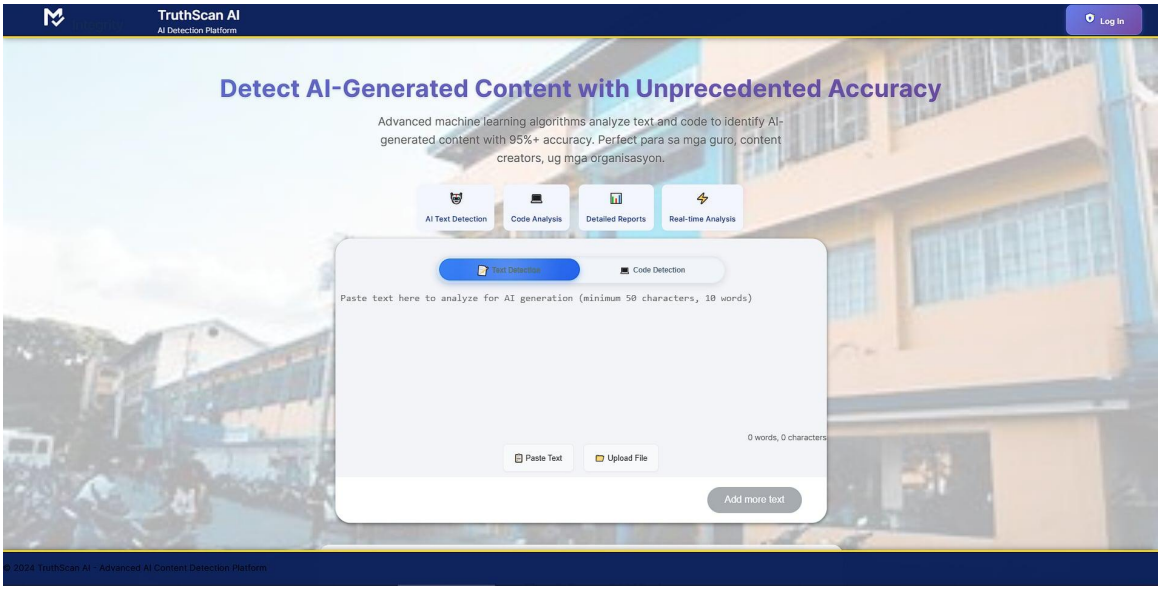


Figure 6: TruthScan AI landing page Text Detection

This page is the primary interface where users can analyze text for AI-generated content. It provides access to AI Text Detection, Detailed Reports, and Real-time Analysis.

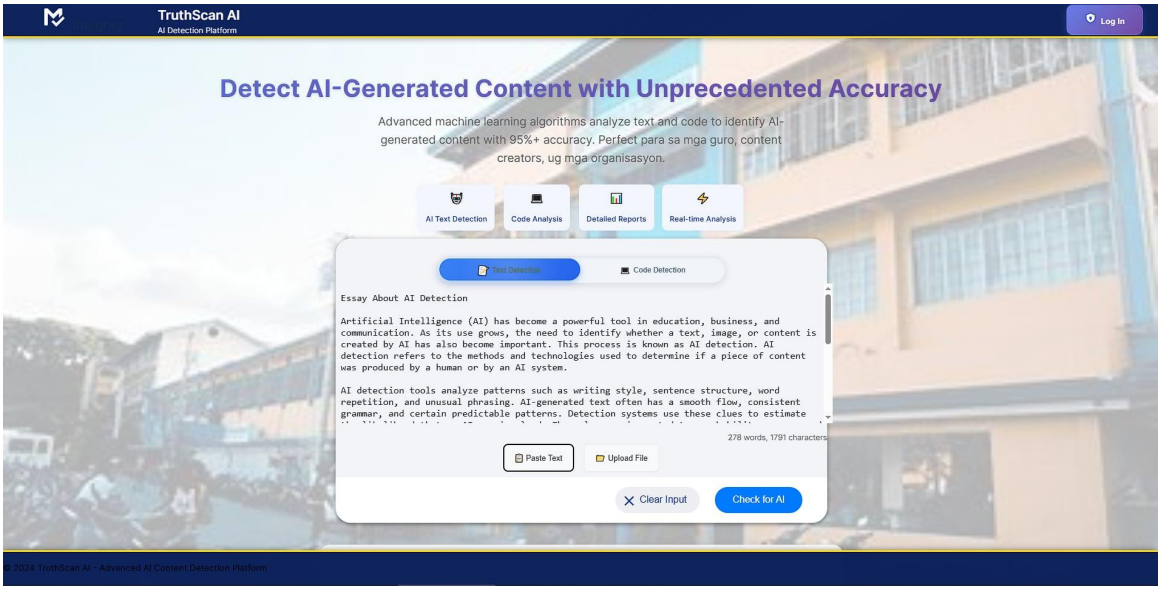


Figure 7: Paste Text on Text Detection function

The “Paste Text” button allows users to make pasting easy by clicking it, where users can simply paste the specific copied text by clicking the button.

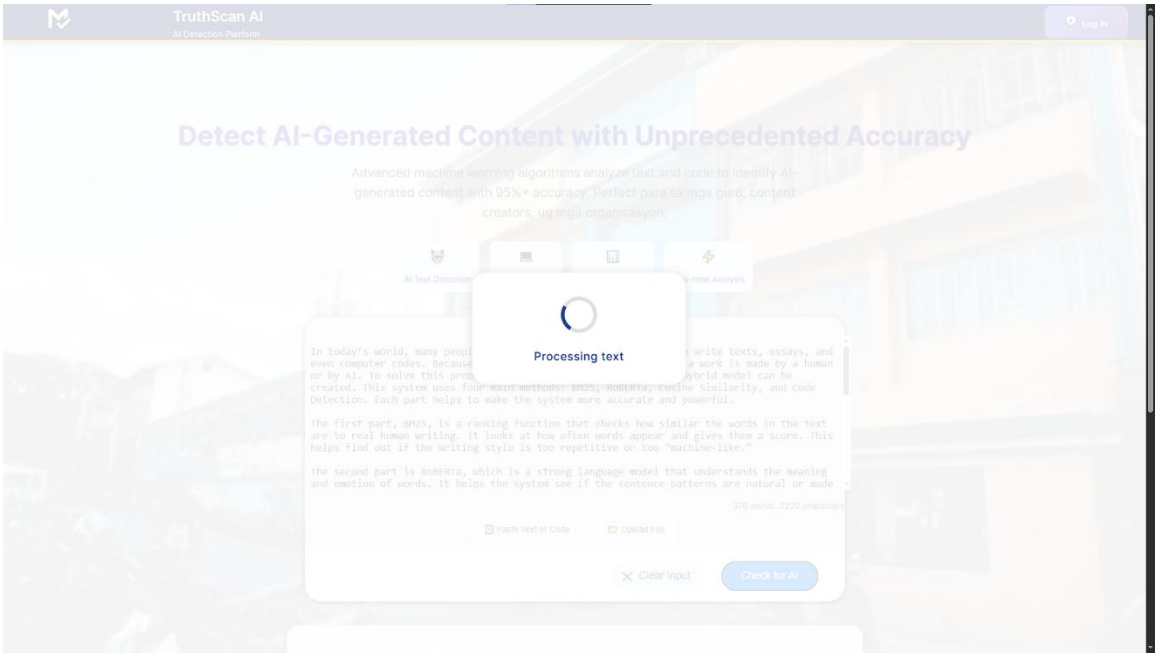


Figure 8: Processing result for Processing text

This section serves as a loading screen when checking the text to determine if it is AI-generated.

It can load fast; it depends on how many words you want to check, and there's no Limit.

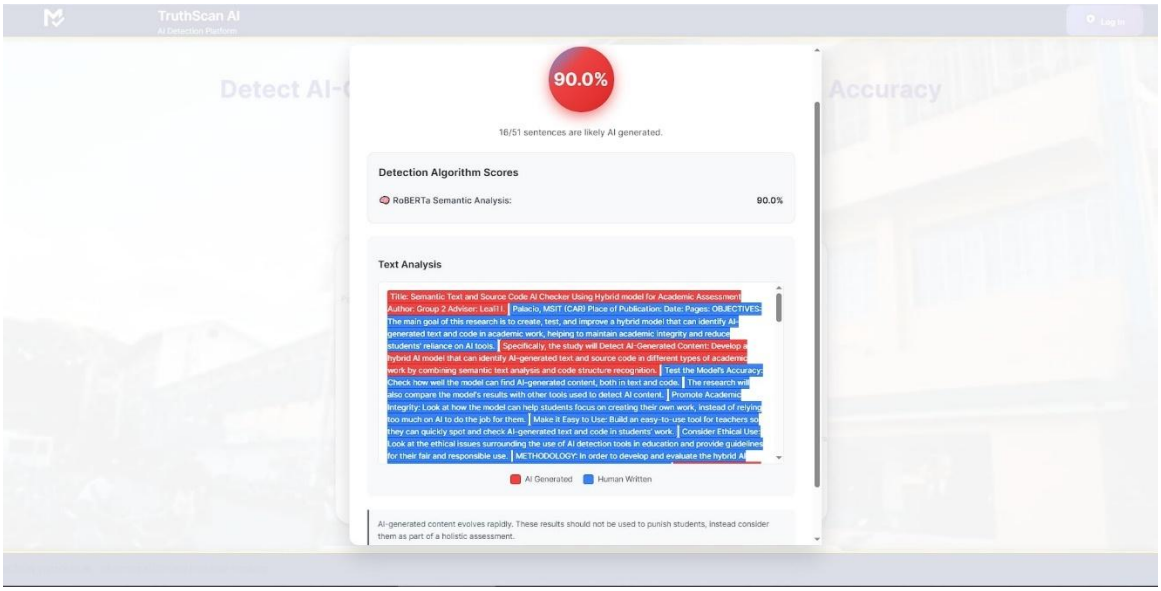


Figure 9: AI text detection scoring

This section provides the score or results with the algorithm, which includes Roberta/GPT classifier, and Hybrid Stylometric + AST Structural Analysis, with the total percentage of the AI-

Generated content highlighted in red, purple, and blue, red for AI-Generated, purple for between the two, and blue for Human-written.

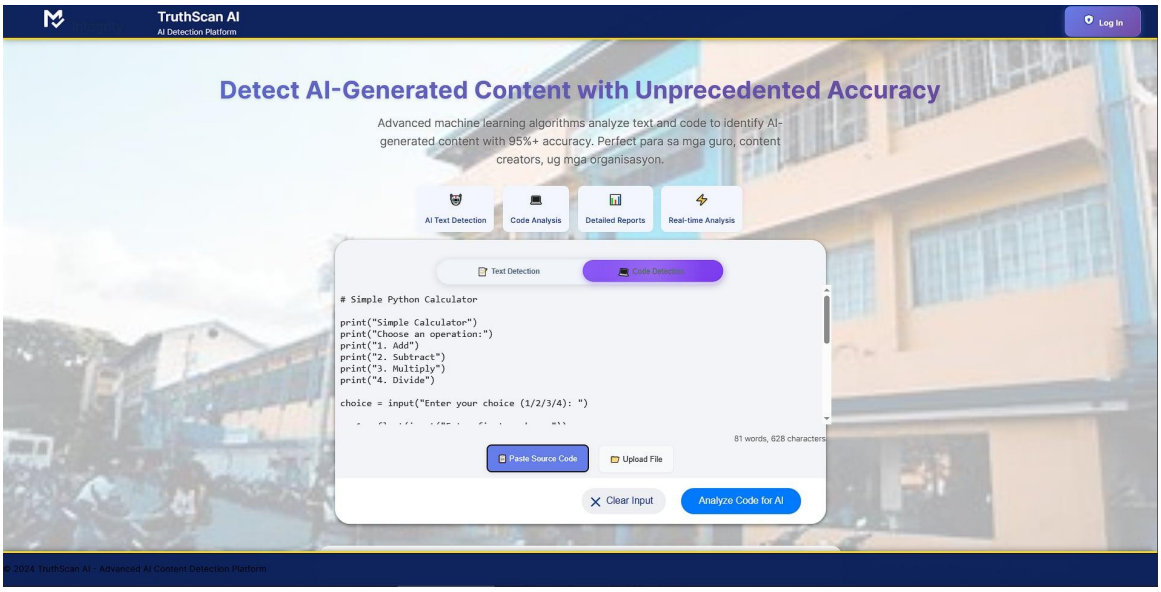


Figure 10: Paste Code on Code Detection function

The “Paste Source Code” button allows users to make pasting easy by clicking it, where users can simply paste the specific copied source code by clicking the button.

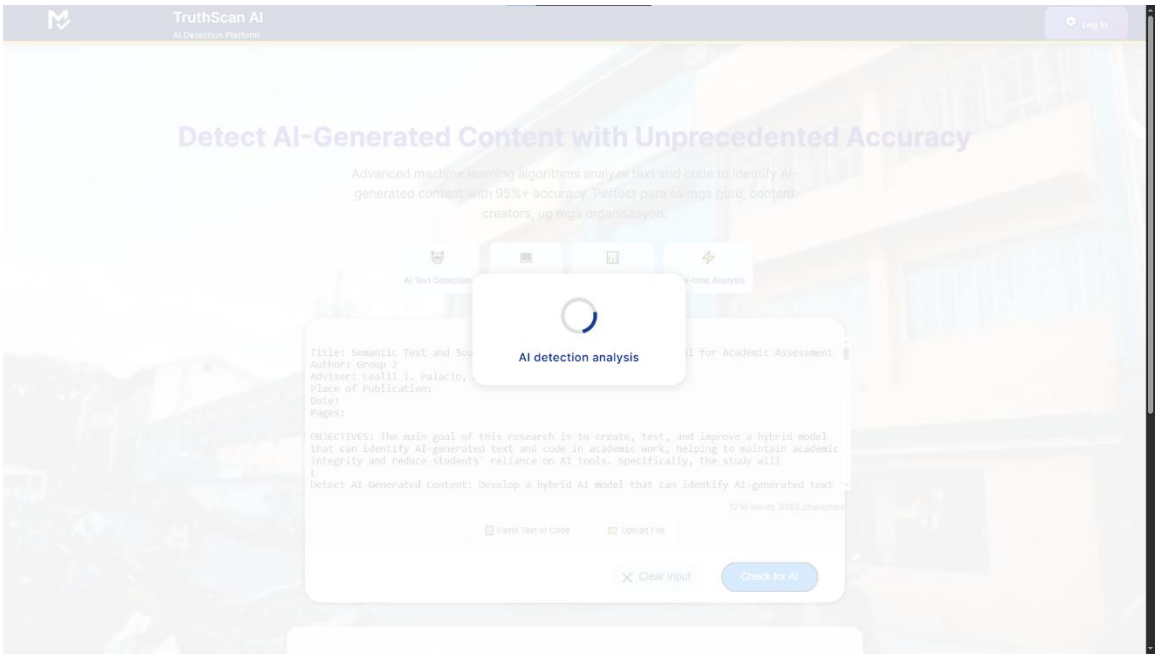




Figure 11: Processing result for AI Source Code Detection Analysis

This section is a loading screen when checking the code and uploaded files, if they are AI-generated. It can load fast; it depends on how many words you want to check, and there's no Limit.

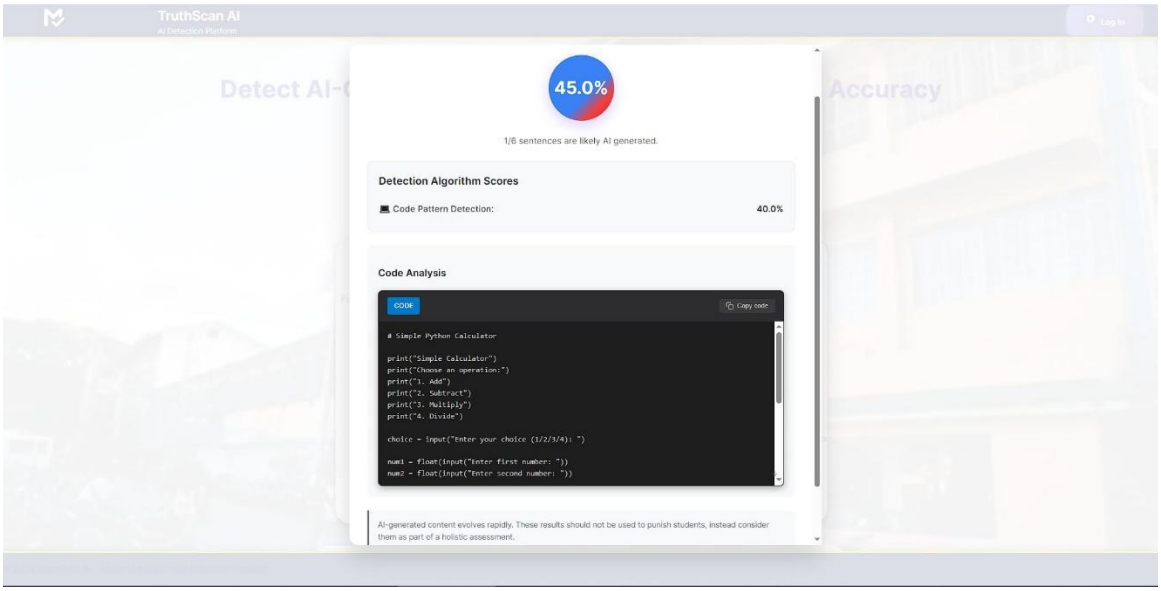


Figure 12: AI Source Code detection scoring

This section provides the score or results with the algorithm, which includes Roberta/GPT classifier, and Hybrid Stylometric + AST Structural Analysis, with the total percentage of the AI-Generated content highlighted in red, purple, and blue, red for AI-Generated, purple for between the two, and blue for Human-written.

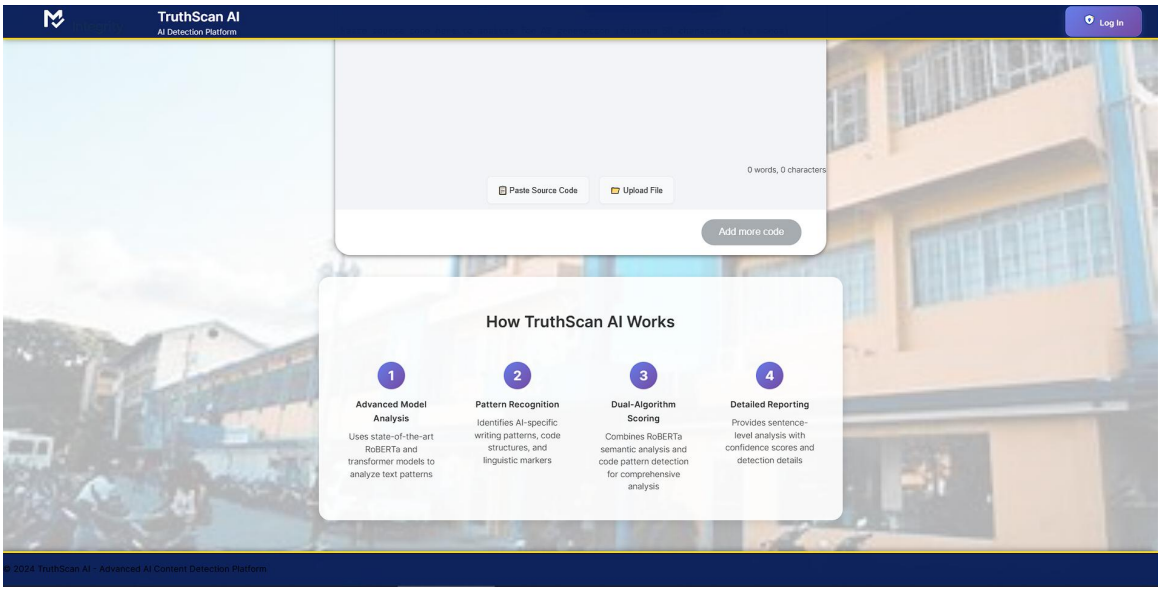


Figure 13: TruthScan AI footer section

This section explains how the TruthScan AI system works, detailing its model analysis, pattern Recognition, multi-algorithm scoring, and detailed reporting features.

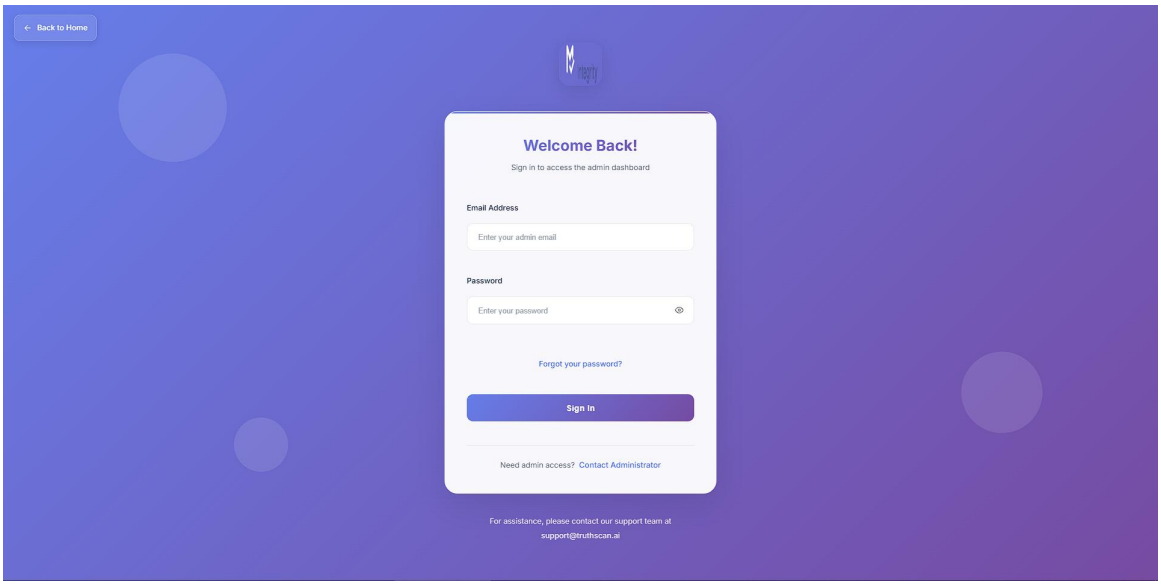


Figure 14: Admin login page

This page allows administrators to securely log in using their credentials to access the TruthScan AI dashboard and manage system Settings.

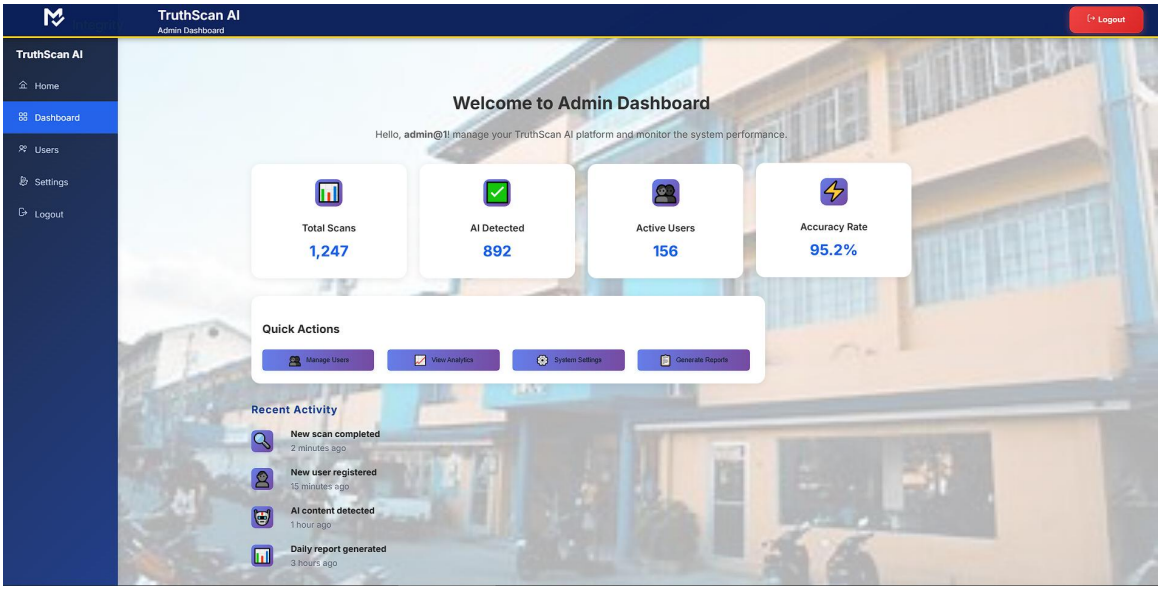


Figure 15: Admin Dashboard

The dashboard displays system statistics such as total scans, AI detections, active users, and overall accuracy rate. It also provides quick actions for managing users, viewing analytics, and Generating reports.

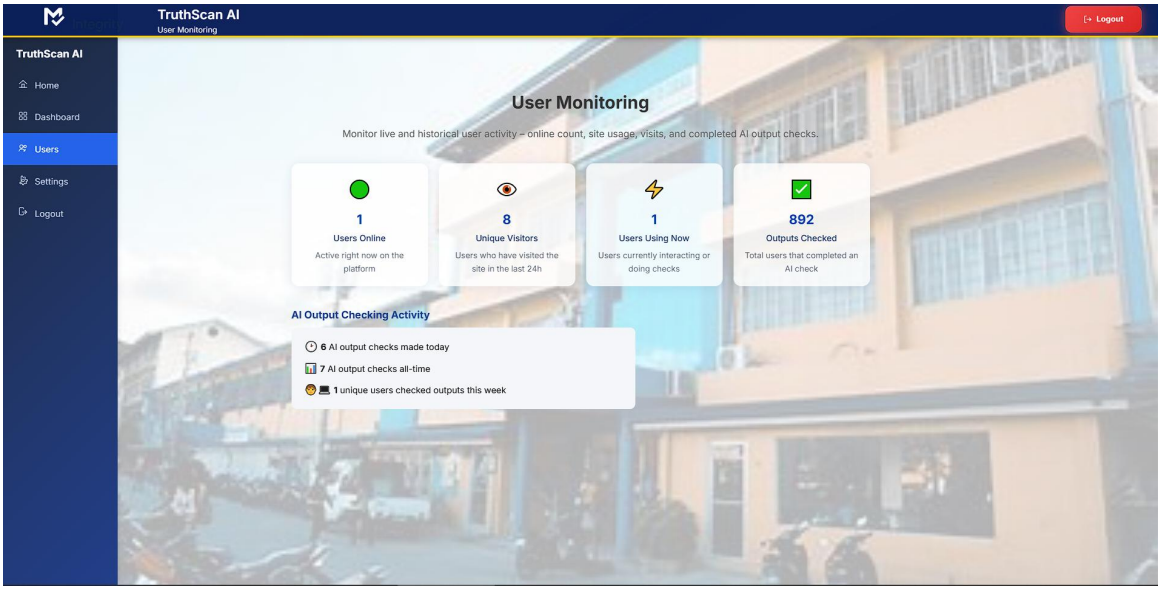


Figure 16: Admin User monitoring

The user monitoring shows how many users are online, unique visitors, users now, outputs, and checked. And also the checking activity.

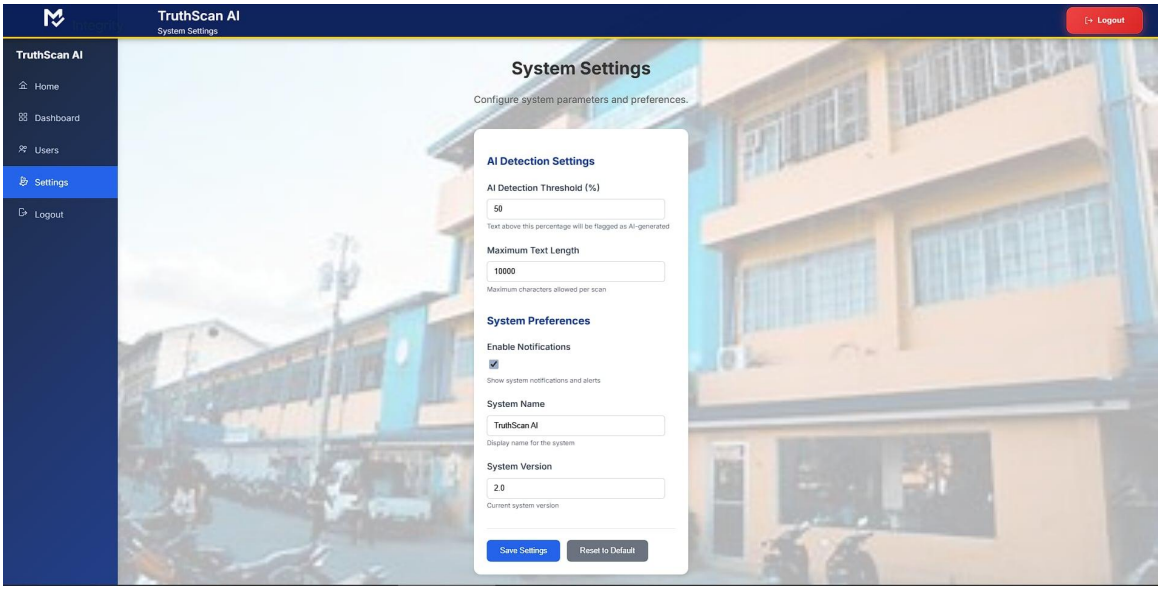


Figure 17: Admin system Settings

This page can change the threshold percentage, the accuracy, and the size of characters that users can Input, the notification, system name, and version. You can update the system using The Settings section.

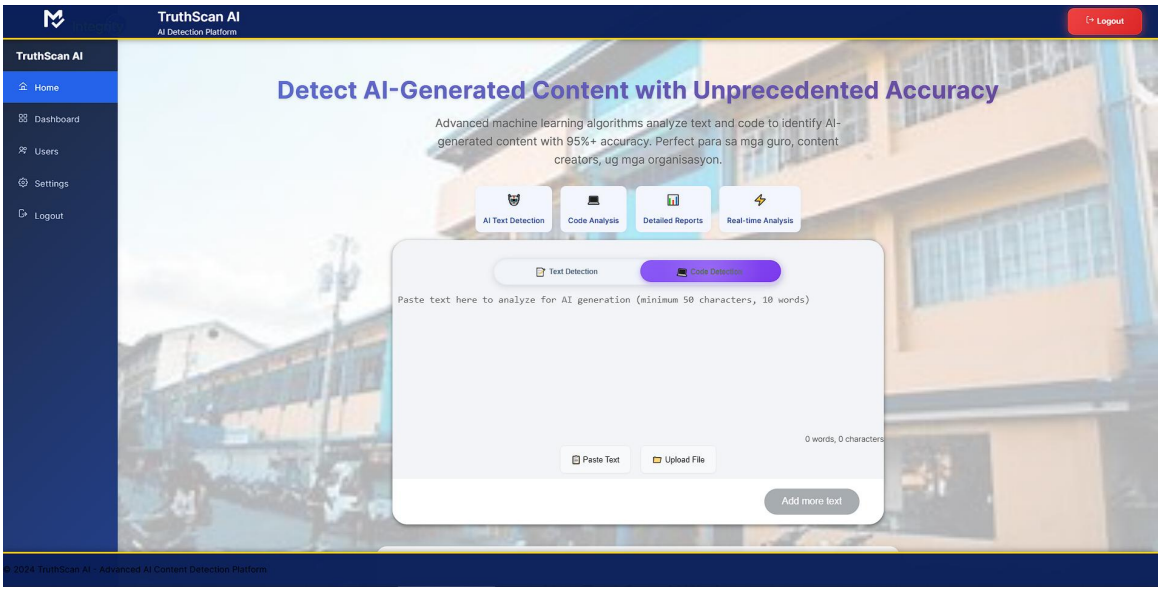


Figure 18: Admin Home Interface

This are the admin's home page view showing navigation options like Home, Dashboard, Users, Settings, and Logout, allowing complete control and monitoring of the AI detection platform.

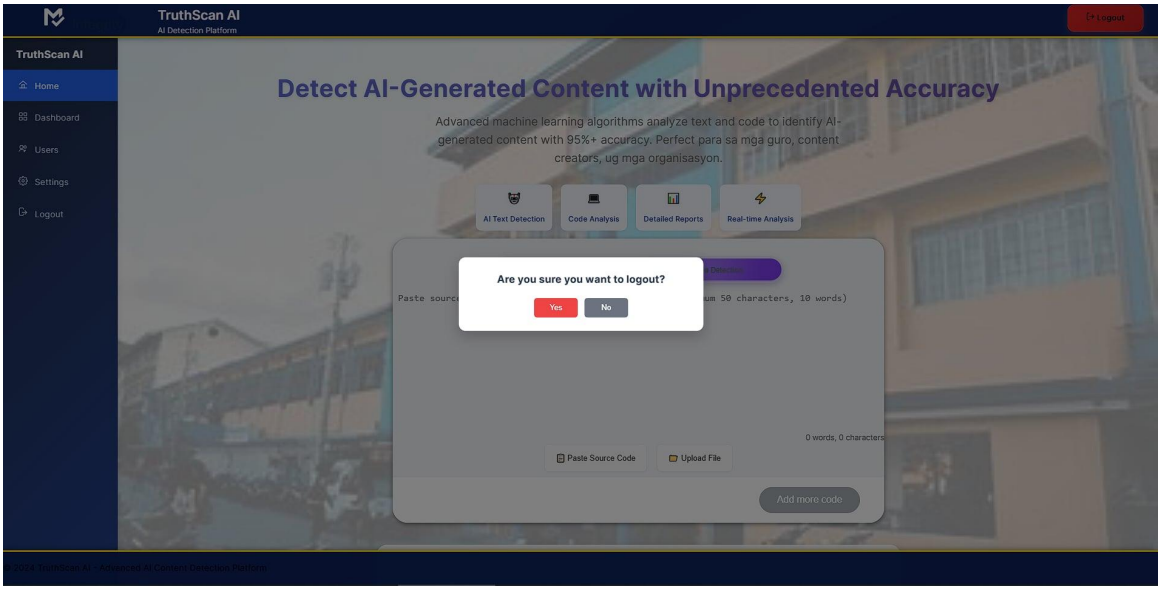


Figure 19: Admin logout button & function

This section has a logout button and a function where the admin can log out.



1.6 Software Platforms, Development Environment, and Tools

Components	Specification	Usage
Internet	Google	A web browser enables users to access and interact with The TruthScan AI platform. It displays the front-end interface where users can Input text or code for AI detection and view the analysis results.

Front-End	HTML, CSS, JavaScript,	The front-end serves as the system's user interface. HTML structures the web pages, CSS styles the design for better visual presentation, and JavaScript provides interactive features.
Back-End	MySQL (Database), Python	The back-end handles data storage and logic. MySQL stores user information, AI detection results, and system logs, while Python manages the application logic, data analysis, and connection between the database and front-end.

Server	Apache	The Apache server, managed through XAMPP, hosts and runs The TruthScan AI web application. It processes requests between the front and back, ensuring smooth communication and Accessibility.

3.8 Hardware Requirements

Components	Specification	Usage
Processor (CPU)	Intel Core i5/Ryzen 5 2400g PRO or higher	The processor performs all logical and computational operations required by the system. It ensures the smooth execution of AI detection algorithms and web-based processes.
Memory (RAM)	Minimum 16 GB (24 GB recommended)	The RAM temporarily stores data and supports multitasking operations, allowing the AI Checker system to process user requests efficiently without lag.
Storage (Hard Drive / SSD)	500 HDD or 500 GB SSD	The storage device holds the system's files, user data, AI models, and database records. SSD provides faster access and improved system performance.
Server Machine	Intel Core i5 Processor (10th Gen or higher), AMD Ryzen 5 2400G Processor or higher, 16 GB, and 500 GB SSD/HDD	The server hosts the web application, manages multiple user connections, and handles AI processing tasks for real-time analysis.
		Provides internet access for

Network / Internet Connection	Stable connection (minimum 20Mbps)	users to connect to the AI Checker web application and enables real-time scanning and report generation.
Graphics (GPU)	Integrated or Dedicated GPU (optional)	Enhances performance in AI model operations, primarily if the system uses deep learning algorithms that require GPU acceleration.
Monitor / Display	15-inch or larger	Displays the system's graphical user interface (GUI) clearly for users and administrators.
Peripheral Devices	Keyboard, Mouse, and Printer (optional)	Allow users and administrators to Input data, navigate the system, and print reports or analysis results.

3.9 EVALUATION MATRIX

EVALUATION AREA	METRIC	DESCRIPTION/CRITERIA
Model Tested	**Hybrid Mode**	An algorithm integrating models like a RoBERTa/GPT classifier and Hybrid Stylometric + AST Structural Analysis
Core Effectiveness	**Accuracy Test**	Measures the model's ability to accurately distinguish between human-written and AI-generated content in text and code.
Accuracy Detail	**False Positives**	Human work mistakenly identified as AI-generated content, an ethical issue that needs to be addressed with more training data
Accuracy Detail	**False Negatives**	AI-generated work mistakenly identified as human work, an ethical issue that needs to be addressed with more training data.
System Reliability	**Functionality Test**	Ensures all system components work as intended, including user functions like "Paste Text or Code" and "Upload File," and admin features like "User Monitoring".
System Efficiency	**Performance Test**	Measures non-functional requirements such as the system's processing speed for analysis and its ability to

		handle inputs of varying lengths
Usability	**User Feedback/Interface**	Assesses if the results are dependable and the interface is straightforward. Feedback suggested enhancing the UI to offer more data interpretation instructions.
Maintenance	**Continuous Testing and Updates**	Necessary to maintain efficacy and accuracy as AI tools advance, integrating new AI writing and coding methodologies23.

2.13 ETHICAL CONSIDERATIONS

The paramount concern in all research, especially when developing technologies like the Semantic Text and Source Code AI Checker, are that it directly impacts humans' subjects and academic Integrity. it involves proactively identifying potential harm, such as unjust accusations from false positives, and implementing protocols to minimize these risks through rigorous testing and a humans-in-the-loop review process.

A. Intellectual Property Rights (IPR)

Intellectual Property Rights (IPR) The developed AI hybrid model, as an original creation, will be protected by IPR owned by the researchers. The research must follow the IPR of any external datasets, open-source libraries, or pre-trained models used in its construction. It are essential to respect all licenses. A straightforward institutional policy are needed to specify who owns the AI system and who holds the intellectual property rights of The Academic submissions

it analyzes, including text and code. This policy makes sure that the system are used Only for assessment. It does not violate the students' original ownership of their work.

B. Informed Consents

Informed Consents: If the AI checker are tested with real student work, you must get consent from all participants, including students and educators. Individuals involved should get straightforward information about the system's goals, the specific data it collects and reviews, and the security measures to protect their information. The consent process must clearly explain that their academic submissions are being used to validate and improve the AI assessment tool. This allows them to participate voluntarily. Because the technology are complex, researchers must make special efforts to ensure non-technical users understand the AI's functionality before consent.

C. Data Privacy and Confidentiality

Data Privacy and Confidentiality Handling sensitive academic data requires the immediate anonymization or de-identification of all student submissions used for training or testing the AI model. This protects their privacy. Strong confidentiality protocols must be set up to ensure that Only authorized personnel, like course instructors or academic administrators, can view the originality check results.

D. Voluntary Participation and Freedom to Withdraw

Voluntary Participation and Freedom to Withdraw: Participation in the validation or testing phase of the AI checker must be entirely voluntary for everyone, including students and faculty. There should be no implicit or explicit pressure to take part. The research design must ensure



that participants can withdraw from the study any time. They should not face any penalty or harm to their academic standing. Straightforward procedures for withdrawal must be set up. These should explain what will happen to the data previously collected when they make a request. This ethical standard ensures that the power dynamic between researchers and students does not undermine the independence of the participants.

E. Minimization of Harm and Risk Management

Minimizing Harm and Managing Risk. The primary ethical risk are the chance of false positives. This happens when the AI wrongly identifies original student work as plagiarized, leading to serious academic consequences. Testing and validation are crucial for achieving the highest accuracy and minimizing errors as much as possible. Researchers must clearly define the boundaries and Limitations of their system. If the AI flagging tool identifies a student's work, there should be a straightforward process for humans review and appeal.

F. Beneficence and Contribution to Knowledge

Beneficence and Contribution to Knowledge The research makes a valuable contribution by creating a hybrid AI model. This model significantly improves the reliability of academic assessments for written and programming assignments. By creating a fair assessment tool, the research helps all students. It shows the importance of genuine intellectual effort in academics. This innovation improves automated plagiarism detection and semantic code analysis, which benefits educational technology.

G. Fairness and Selecting Participants

The rule of justice requires that we share the risks and benefits of the research reasonably. This ensures that no specific group are unfairly targeted or excluded. Researchers must examine the entire system to ensure fairness. The data used to teach and test the system should be diverse and reflect the different people who will use it across various fields and programming languages. This helps prevent the AI from being biased. Researchers need to test the system actively to ensure the AI does not unfairly penalize certain writing styles, cultural backgrounds, or programming habits. The final version of the AI checker must be used reasonably and consistently for all students. This approach will help create an equitable testing environment.

H. Good Data and Accuracy

The quality of the data used to teach and test the AI are essential, paramount, critical, crucial If the data has any mistakes or are biased (unfair), it will hurt how fair and beneficial the AI checker.

We must have straightforward rules for gathering, safely storing, and handling the data to keep all academic information real and complete. All statements about how accurate the system are must be reported honestly and clearly. This includes performance numbers like how often it makes a mistake by saying something are wrong when it's right (False Positives) and saying something are right when it's wrong (False Negatives) for both text and code. We need strong ways to check the system to make sure its results are reliable and trustworthy for making academic decisions.

I. Being straightforward and Honest in Your Report

The research report must fully show the methods used. This means explaining the specific kind of AI model, the computer algorithms (rules) it follows, and the primary weaknesses of the AI checker. Researchers must report all the results, including both the good stuff (successes) and the bad stuff (failures) that happened while creating and testing the AI against humans grading.

You must clearly state any potential conflicts of interest (situations where someone's personal interests might affect the results), any money received (financial support), or any work done with other organizations (institutional collaborations) in the final paper or book.

This openness (transparency) makes sure the system can be Checked (audited). This allows students and professors to understand and trust how the AI makes its grading decisions**.**

J. Using Protected or Owned Materials

The research team must follow Fair Use rules or get official permission when they use materials that belong to others (copyrighted materials). This includes stuff like specific programming guides or published school papers used in the training data. A required check of all parts from other sources (third-party components), including software tools and algorithms, must confirm that using them follows their specific licenses (rules for use), whether they are free (open-source) or paid (commercial). If the system uses any protected technology (patented technology) for its primary functions, they must get official permission (licensing) from the owner of the patent before they start using it. Breaking copyright or patent laws are a grave ethical and legal mistake that must be avoided.

K. Thinking About Humans Students

Even though this research are mainly about computer programs (software), the student work (academic submissions) being looked at contains information from people (indirect humans subjects). This requires that we use the same ethical protections usually given to people in studies. Researchers must make sure that using old student assignments to teach the AI are fair (justified) and that this use won't hurt a student's chances now or in the future.

L. Official Permission and School Approval

Because this research involves sensitive student information and an AI tool used for grading schoolwork, getting official Ethical Clearance and approval from the Institutional Review Board (IRB) are necessary. The complete research plan, which includes the plan for handling data, the consent forms (permission slips), and the check for risks, must be given to the school's IRB to look at.

The research cannot start until the proper school group gives an official Certificate of Research Approval. Researchers must stick to the approved plan and must ask the IRB for changes (amendments) If it make any significant changes to how the study are set up.

M. Using AI and Other Technology Responsibly

When building this mixed AI checker, we must follow the rules of Responsible AI. This means The tool must be beneficial and helpful, and it must not cause or increase unfairness (bias) in grading schoolwork. The system should be made to be straightforward and easy to understand, moving



away from a 'black-box' model (where you can't see how it works). This way, teachers can see why An AI flags something as plagiarism. Researchers must keep checking for fairness (ongoing fairness audits) across different student groups to lower unfair treatment by the computer rules (algorithmic discrimination) and make sure The tool are used reasonably.

Responsible use also includes planning to keep An AI model maintained and updated for a long time. This are needed to make sure it stays accurate and relevant as school standards change.

CHAPTER 4

SOFTWARE DEVELOPMENT AND TESTING



4.1 Development Process

4.2 Testing Process

- Accuracy Test

-Functionality Test

-Performance Test

CHAPTER 5

SUMMARY, CONCLUSION, RECOMMENDATION,

5.1. Summary of Findings

5.2. Conclusion

5.3. Recommendation/s

DELETE BOX AFTER READING

Check the guidelines in research writing.



REFERENCES

[1] Q. Wang, "The use of semantic similarity tools in automated content scoring of fact-based essays written by EFL learners," *Educ Inf Technol (Dordr)*, vol. 27, no. 9, pp. 13021–13049, Nov. 2022, doi: 10.1007/S10639-022-11179-1/TABLES/6.

[2] F. D. Pereira *et al.*, "Towards a Human-AI Hybrid System for Categorising Programming Problems," *SIGCSE 2021 - Proceedings of*

the 52nd ACM Technical Symposium on Computer Science Education, pp. 94–100, Mar. 2021, doi: 10.1145/3408877.3432422.

[3] R. Verma and P. K. [Guided by] Gupta, "detecting," 2023, Accessed: Oct. 08, 2025. [Online]. Available: http://www.ir.juit.ac.in:8080/jspui/jspui/handle/123456789/10009

[4] Z. Xu and V. S. Sheng, "Detecting AI-Generated Code Assignments Using Perplexity of Large Language Models," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 21, pp. 23155–23162, Mar. 2024, doi: 10.1609/AAAI.V38I21.30361.

[5] Y. Feng, S. Vanam, M. Cherukupally, W. Zheng, M. Qiu, and H. Chen, "Investigating Code Generation Performance of ChatGPT with Crowdsourcing Social Data," *Proceedings - International Computer Software and Applications Conference*, vol. 2023-June, pp. 876–885, 2023, doi: 10.1109/COMPSAC57700.2023.00117.

[6] "Expert-Authored and Machine-Generated Short-Answer Questions for Assessing Students Learning Performance on JSTOR." Accessed: Oct. 08, 2025. [Online]. Available: https://www.jstor.org/stable/27032863

[7] P. Vittorini, S. Menini, and S. Tonelli, "An AI-Based System for Formative and Summative Assessment in Data Science Courses," *Int J Artif Intell Educ*, vol. 31, no. 2, pp. 159–185, Jun. 2021, doi: 10.1007/S40593-020-00230-2/TABLES/5.

[8] I. Papachristou, G. Dimitroulakos, and C. Vassilakis, "Automated Test Generation and Marking Using LLMs," *Electronics 2025, Vol. 14, Page 2835*, vol. 14, no. 14, p. 2835, Jul. 2025, doi: 10.3390/ELECTRONICS14142835.

[9] D. M. Salama and N. M. El-Gohary, "Semantic Text Classification for Supporting Automated Compliance Checking in Construction," *Journal of Computing in Civil Engineering*, vol. 30, no. 1, p. 04014106, Feb. 2013, doi: 10.1061/(ASCE)CP.1943-5487.0000301.

[10] S. Tahvili, L. Hatvani, M. Felderer, W. Afzal, and M. Bohlin, "Automated functional dependency detection between test cases using Doc2Vec and Clustering," *Proceedings - 2019 IEEE International Conference on Artificial Intelligence Testing, AITest 2019*, pp. 19–26, May 2019, doi: 10.1109/AITEST.2019.00-13.

[11] G. Xu, L. Liu, and J. Dong, "Vulnerability Detection of Ethereum Smart Contract Based on SolBERT-BiGRU-Attention Hybrid Neural Model.," *Comput Model Eng Sci*, vol. 137, no. 1, p. 903, Oct. 2023, doi: 10.32604/CMES.2023.026627.

[12] S. A. Chatzichristofis, A. Tsopozidis, A. Kyriakidou-Zacharoudiou, S. Evripidou, and A. Amanatiadis, "Designing an AI-Supported Framework for Literary Text Adaptation in Primary Classrooms," *AI 2025, Vol. 6, Page 150*, vol. 6, no. 7, p. 150, Jul. 2025, doi: 10.3390/AI6070150.

[13] A. Amirzhanov, C. Turan, and A. Makhmutova, "Plagiarism types and detection methods: a systematic survey of algorithms in text analysis," *Front Comput Sci*, vol. 7, p. 1504725, Mar. 2025, doi: 10.3389/FCOMP.2025.1504725/BIBTEX.

[14] "Integrated Technique of Natural Language Texts and Source Codes Authorship Verification in the Academic Environment," *ieeexplore.ieee.orgA Romanov, A Kurtukova, A Fedotova, A ShelupanovIEEE Access, 2025•ieeexplore.ieee.org*, Accessed: Oct. 08,

2025. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/11059894/

[15] "A Hybrid Approach for Automatic Question Generation from Program Codes. | EBSCOhost." Accessed: Oct. 08, 2025. [Online]. Available: https://openurl.ebsco.com/EPDB%3Agcd%3A8%3A12610712/detailv2?sid=ebsco%3Aplink%3Ascholar&id=ebsco%3Agcd%3A175401573&crl=c&link_origin=scholar.google.com

[16] S. Pudasaini, L. Miralles-Pechuán, D. Lillis, and M. Llorens Salvador, "Survey on AI-Generated Plagiarism Detection: The Impact of Large Language Models on Academic Integrity," *J Acad Ethics*, vol. 23, no. 3, pp. 1137–1170, Sep. 2024, doi: 10.1007/S10805-024-09576-X/METRICS.

[17] M. N. Alpdemir, Y. Alpdemir, and S. Doğan, "An Automated Hybrid Exam Evaluation Framework for Textual Courses Using AI," *Lecture Notes on Data Engineering and Communications Technologies*, vol. 228, pp. 121–134, 2025, doi: 10.1007/978-981-97-9255-9_9.

[18] T. Foltýnek, N. Meuschke, and B. Gipp, "Academic Plagiarism Detection," *ACM Computing Surveys (CSUR)*, vol. 52, no. 6, Oct. 2019, doi: 10.1145/3345317.

[19] L. Batista, D. Apostolou, A. Mesquita, W. Pang, and Z. Wei, "Shaping the Future of Higher Education: A Technology Usage Study on Generative AI Innovations," *Information 2025, Vol. 16, Page 95*, vol. 16, no. 2, p. 95, Jan. 2025, doi: 10.3390/INFO16020095.

[20] M. Qorich, R. E. O.-E. S. with Applications, and undefined 2025, "Detection of Artificial Intelligence-Generated Essays for Academic Assessment Integrity using Large Language Models," *ElsevierM Qorich*,

R El OuazzaniExpert Systems with Applications, 2025•Elsevier, Accessed: Oct. 08, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S095741742502024X

[21] O. Karnalim, H. Toba, and M. C. Johan, "Detecting AI assisted submissions in introductory programming via code anomaly," *Educ Inf Technol (Dordr)*, vol. 29, no. 13, pp. 16841–16866, Sep. 2024, doi: 10.1007/S10639-024-12520-6/METRICS.

[22] M. Zakeri-Nasrabadi, S. Parsa, M. Ramezani, C. Roy, and M. Ekhtiarzadeh, "A systematic literature review on source code similarity measurement and clone detection: Techniques, applications, and challenges," *Journal of Systems and Software*, vol. 204, p. 111796, Oct. 2023, doi: 10.1016/J.JSS.2023.111796.

[23] B. Alhijawi, R. Jarrar, A. AbuAlRub, and A. Bader, "Deep learning detection method for large language models-generated scientific content," *Neural Comput Appl*, vol. 37, no. 1, pp. 91–104, Jan. 2025, doi: 10.1007/S00521-024-10538-Y/METRICS.

[24] M. Qorich and R. El Ouazzani, "Detection of artificial intelligence-generated essays for academic assessment integrity using large language models," *Expert Syst Appl*, vol. 291, p. 128405, Oct. 2025, doi: 10.1016/J.ESWA.2025.128405.

[25] "A Generative AI-Driven Method-Level Semantic Clone Detection Based on the Structural and Semantical Comparison of Methods | IEEE Journals & Magazine | IEEE Xplore." Accessed: Oct. 08, 2025. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10531717

[26] W. Brach, K. Kost'al, and M. Ries, "Can Large Language Model Detect Plagiarism in Source Code?," *2024 2nd International Conference on Foundation and Large Language Models, FLLM 2024*, pp. 370–377, 2024, doi: 10.1109/FLLM63129.2024.10852497.

[27] V. H. Kalmani, A. C. Adamuthe, A. P. Gondil, V. P. Patil, R. A. Kore, and V. M. Metkari, "AI vs. Human Writing: Developing a Novel Method for Text Authenticity Detection in Education," *International Journal of Modern Education and Computer Science*, vol. 17, no. 3, pp. 45–58, Jun. 2025, doi: 10.5815/IJMECS.2025.03.04.

[28] "Semantic clone detection without type limitations." Accessed: Oct. 12, 2025. [Online]. Available: https://www.politesi.polimi.it/handle/10589/227895

[29] F. Ebrahim and M. Joy, "Semantic Similarity Search for Source Code Plagiarism Detection: An Exploratory Study," *Annual Conference on Innovation and Technology in Computer Science Education, ITiCSE*, vol. 1, pp. 360–366, Jul. 2024, doi: 10.1145/3649217.3653622.

[30] S. Kumar, S. Garg, S. Sengupta, T. Ghosal, and A. Ekbal, "MixRevDetect: Towards Detecting AI-Generated Content in Hybrid Peer Reviews.," vol. 2, pp. 944–953, Jun. 2025, doi: 10.18653/V1/2025.NAACL-SHORT.79.

[31] S. Jimenez, S. P. Cucerzan, F. A. Gonzalez, A. Gelbukh, and G. Dueñas, "BM25-CTF: Improving TF and IDF factors in BM25 by using collection term frequencies," *Journal of Intelligent & Fuzzy Systems*, vol. 34, no. 5, pp. 2887–2899, 2018, doi: 10.3233/JIFS-169475.

[32] S. Imtiaz and E. K. Hashi, "Analyzing the Performance of Sentiment Analysis Using BERT, DistilBERT, RoBERTa and GPT-2," *2024 6th*

International Conference on Sustainable Technologies for Industry 5.0, STI 2024, 2024, doi: 10.1109/STI64222.2024.10951049.

[33] "Malicious Code Detection under 5G HetNets Based on a Multi-Objective RBM Model | IEEE Journals & Magazine | IEEE Xplore." Accessed: Oct. 14, 2025. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9318423

[34] A. Trotman, A. Puurula, and B. Burgess, "Improvements to BM25 and language models examined," ACM International Conference Proceeding Series, vol. 27-28-November-2014, pp. 58–65, Nov. 2014, doi: 10.1145/2682862.2682863.

[35] A. Sak, "Using cosine similarity classifier for NLP analysis in construction field texts," AIP Conf Proc, vol. 2791, no. 1, Aug. 2023, doi: 10.1063/5.0143607.

[36] T. Rasul et al., "Enhancing academic integrity among students in GenAI Era:A holistic framework," International Journal of Management Education, vol. 22, no. 3, Nov. 2024, doi: 10.1016/j.ijme.2024.101041.

[37] D. Watson, "Source Code Stylometry and Authorship Attribution for Open Source".

[38] G. O. Adebayo, "Multimodal stylometry: A novel approach for authorship identification.," Electronic Theses and Dissertations, May 2024, doi: 10.18297/etd/4361.

[39] A. Gurioli, M. Gabbrielli, and S. Zacchiroli, "Stylometry for real-world expert coders: a zero-shot approach," PeerJ Comput Sci, vol. 10, p. e2429, Nov. 2024, doi: 10.7717/PEERJ-CS.2429.

[40] D. R. Amancio, "A Complex Network Approach to Stylometry," *PLoS One*, vol. 10, no. 8, p. e0136076, Aug. 2015, doi: 10.1371/JOURNAL.PONE.0136076.

[41] G. O. Adebayo, "Multimodal stylometry: A novel approach for authorship identification.," *Electronic Theses and Dissertations*, May 2024, doi: 10.18297/etd/4361.

[42] M. Smytzek, M. Eberlein, L. Grunske, and A. Zeller, "jAST: Analyzing and Modifying Java ASTs with Python," *Proceedings of the ACM SIGSOFT Symposium on the Foundations of Software Engineering*, pp. 1143–1147, Jul. 2025, doi: 10.1145/3696630.3728598;PAGE:STRING:ARTICLE/CHAPTER.

[43] K. Dimitrios, A. Apostolos, C. Alexander, and X. Stylianos, "Development of a system for detecting refactorings in software code history," 2025, Accessed: Dec. 05, 2025. [Online]. Available: http://dspace.lib.uom.gr/handle/2159/33815

[44] K. Yang *et al.*, "An Extensive Study of the Structure Features in Transformer-based Code Semantic Summarization," *IEEE International Conference on Program Comprehension*, vol. 2023-May, pp. 89–100, 2023, doi: 10.1109/ICPC58990.2023.00024.

[45] D. Ribeiro dos Santos, "Static Analysis by Abstract Interpretation Collecting Types of Python Programs," p. 255, Sep. 2018, doi: 10.34894/VQ1DJA.

[46] K. Shahnazari, M. Keshtparvar, and S. M. Ayyoubzadeh, "PARSI: Persian Authorship Recognition via Stylometric Integration," Jun. 2025, Accessed: Dec. 05, 2025. [Online]. Available: https://arxiv.org/pdf/2506.21840

CURRICULUM VITAE

Recent ID picture (size 1 ½ x 2, preferably

NAME:
ADDRESS:
EMAIL:
Personal Information
Sex :
Date of birth :
Height :
Weight :
Civil Status :
Educational Attainment
Elementary :
Secondary :
Tertiary :
Membership/Affiliations

ORGANIZATION	POSITIONS HELD

CURRICULUM VITAE

Recent ID picture (size 1 ½ x 2,

NAME:
ADDRESS:
EMAIL:
Personal Information

Sex :
Date of birth :
Height :
Weight :
Civil Status :

Educational Attainment

Elementary :
Secondary :
Tertiary :

Membership/Affiliations

ORGANIZATION	POSITIONS HELD

APPENDICES

A – Letter of Approval with Certificate of Appearance and Photo
B – Instrument with Informed Consent
C – Map of the Research Locale *(Include Coordinates)*
D – Curriculum Vitae
E – Narrative & Photo Documentation
F– Ethics Committee Approval

CERTIFICATE OF AUTHENTIC AUTHORSHIP

This is to certify that I/we, the undersigned author(s), have solely and honestly written the entire paper titled: "Click here to enter text." All contents, ideas, data, and analysis presented in this paper result from my/our own diligent work and intellectual effort. The paper has been crafted to the best of my/our abilities, in full compliance with academic Integrity standards and ethical research practices.

I/We further declare that:

1. This paper is free from plagiarism or copyright infringement.

2. All referenced materials have been properly cited, and due credit has been given to the original authors and sources.
3. The work contains no falsified or fabricated data or information.

I/We acknowledge that if any rules or laws related to copyright, plagiarism, or other related legal or ethical standards are violated, Saint Michael College of Caraga shall not be held accountable for my/our actions. Any legal, academic, or moral consequences arising from such violations shall solely be my/our responsibility.

Signed this
Date: Click here to enter text.
Location: Click here to enter text.

Author(s):

Click here to enter text.
Author

Click here to enter text.
Author

Click here to enter text.
Author

Click here to enter text.
Author